MINISTRY OF EDUCATION AND TRAINING

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING

FACULTY OF ADVANCED EDUCATION

FINAL REPORT

SOCIAL MEDIA WEBSITE

SUBJECT: WEB PROGRAMMING

CLASS: Web Programming_ Group 02FIE

INSTRUCTOR: Ms. Nguyen Thuy An

GROUP: 12

MEMBERS OF THE GROUP: Huỳnh Vũ Minh Hiền – 24110091

Nguyễn Quốc Bảo Khang – 24110099

Lê Sĩ Nhật Khuê – 24110102

Ngô Quang Trường – 24110142

Ho Chi Minh City, October 2026

TOPIC: Social Media Website

No.	Fullname	ID Student	Completion Level
1	Huỳnh Vũ Minh Hiền	24110091	100%
2	Nguyễn Quốc Bảo Khang	24110099	100%
3	Lê Sĩ Nhật Khuê	24110102	100%
4	Ngô Quang Trường	24110142	100%

- *Team Leader: Ngô Quang Trường – Phone: 0838282133*

Instructor's feedback:

.....

.....

.....

.....

.....

TABLE OF CONTENTS

CHAPTER 1. PROJECT OVERVIEW 1
1.1. Project Introduction 1
1.2. Objectives 1
1.3. Target Users 1
1.4. Overall Architecture 1
CHAPTER 2. TECHNOLOGIES USED 3
2.1. Platform and MVC Architecture 3
2.2. Data Access 4
2.3. User Interface 4
2.4. Real-time Communication 5
2.5. Security 5
2.6. Artificial Intelligence 6
2.7. Biometric Authentication and Digital Identity 6
2.8. Media Storage on Cloudinary 6
2.9. Deployment Infrastructure 7
CHAPTER 3. SYSTEM FEATURES 9
3.1. Registration, Login, and Password Recovery 9
3.2. Newsfeed 10
3.3. Stories 14
3.4. Profile Page 14
3.5. Friends 15
3.6. Messaging 16
3.7. Notifications 16
3.8. Groups 17
3.9. Videos and Saved Posts 18
3.10. Accent-Insensitive Search 19
3.11. AI Assistant 20
3.12. Settings 22
3.13. Passkeys and Digital Identity 23
3.14. Report Violations 24
3.15. Administration 24
3.16. Terms and Policies 26
CHAPTER 4. DATABASE 28
4.1. DBMS and Connection Method 28

4.2. Database Structure 29
4.3. Subsystem Relationship Diagrams 30
4.3.1. Users and Security 30
4.3.2. Posts and Interactions 30
4.3.3. Friends and Groups 32
4.3.4. Messaging and Notifications 32
4.3.5. Stories, Reports, and Administration 34
4.4. General Constraints 36
4.5. Current Data 36
4.6. Backup and Recovery 36
CHAPTER 5. CONCLUSION 38
5.1. Results Achieved 38
5.2. Limitations 38
5.3. Future Development 38
REFERENCES 39

TABLE OF FIGURES

Figure 1.1. Overall architecture diagram of UTErSpace 2
Figure 2.1. UTErSpace media library on Cloudinary 7
Figure 3.1. Login page 9
Figure 3.2. Account registration page 9
Figure 3.3. Password recovery using OTP 10
Figure 3.4. Three-column grid newsfeed 11
Figure 3.5. Single-post feed view 11
Figure 3.6. Post detail and comments modal 12
Figure 3.7. New post creation box 12
Figure 3.8. Shared post with quote commentary 13
Figure 3.9. Hashtag filter page 13
Figure 3.10. Story viewer 14
Figure 3.11. User profile page 15
Figure 3.12. Friends page 15
Figure 3.13. Chat and messaging interface 16
Figure 3.14. Notifications page 17
Figure 3.15. Groups list 17
Figure 3.16. Group detail page 18
Figure 3.17. Video page 19
Figure 3.18. Saved posts page 19
Figure 3.19. Instant search suggestions while typing 20
Figure 3.20. Search results page 20
Figure 3.21. AI assistant generating captions 21
Figure 3.22. Gentle reminder when posting negative content 21
Figure 3.23. Privacy settings 22
Figure 3.24. Personal content filter 22
Figure 3.25. Passkey management 23
Figure 3.26. Linked digital identities 23
Figure 3.27. Report violation dialog 24
Figure 3.28. Admin dashboard overview 25
Figure 3.29. User management 25
Figure 3.30. Handling violation reports 25
Figure 3.31. Admin audit logs 26
Figure 3.32. Post management with negative sentiment filter 26

Figure 3.33. Terms of service page 27
Figure 4.1. Connection configuration in persistence.xml 28
Figure 4.2. JPAUtil class initializing connections 29
Figure 4.3. Users and Security subsystem diagram..... 30
Figure 4.4. Posts and Interactions subsystem diagram..... 31
Figure 4.5. Friends and Groups subsystem diagram 32
Figure 4.6. Messaging and Notifications subsystem diagram..... 34
Figure 4.7. Stories, Reports, and Administration subsystem diagram 35
Figure 4.8. Post records with sentiment labels and Cloudinary paths..... 36
Figure 4.9. Row counts of major database tables..... 36

CHAPTER 1. PROJECT OVERVIEW

1.1. Project Introduction

UTErSpace is a dedicated social network for students of Ho Chi Minh City University of Technology and Education (HCMUTE). Students use UTErSpace to share posts, exchange study materials, join study groups and clubs, and message friends within an exclusive campus community. The application is built using Java Servlets and JSP, runs on Apache Tomcat 9, stores data in MySQL 8.4, and hosts images and videos on Cloudinary.

1.2. Objectives

The project aims to achieve three main goals. First, strictly apply the multi-tier MVC architecture of Java EE 8 so that the codebase remains modular, clean, and easy to maintain. Second, provide all core social networking features, including a newsfeed, friends, real-time messaging, groups, stories, and administration. Third, integrate advanced technologies such as artificial intelligence, accent-insensitive Vietnamese search, biometric authentication, and decentralized digital identity, while ensuring robust data security for users.

1.3. Target Users

The system serves three user groups. Unauthenticated guests can only access the login and registration pages, terms of service, privacy policy, and the digital credential verification page. Regular users have full access to all social networking features. Administrators work exclusively within the Admin area: upon logging in, admin accounts are routed directly to the /admin page and cannot access the regular user interface, preventing administrative privileges from mingling with personal activities.

1.4. Overall Architecture

Every browser request passes through six processing layers. The browser renders JSP pages and sends requests via HTML forms or the fetch API, accompanied by a CSRF anti-forgery token. A chain of four Filters sequentially enforces UTF-8 character encoding, validates authentication, checks administrative privileges, and verifies CSRF tokens. The Controller layer consists of 22 Servlets that receive requests, validate input data, and enforce access controls. The Service layer contains shared business logic, while the DAO

layer queries the database via JPA and Hibernate. At the final tier, data is stored in MySQL, media files are hosted on Cloudinary, and real-time events are pushed to the browser via WebSocket.

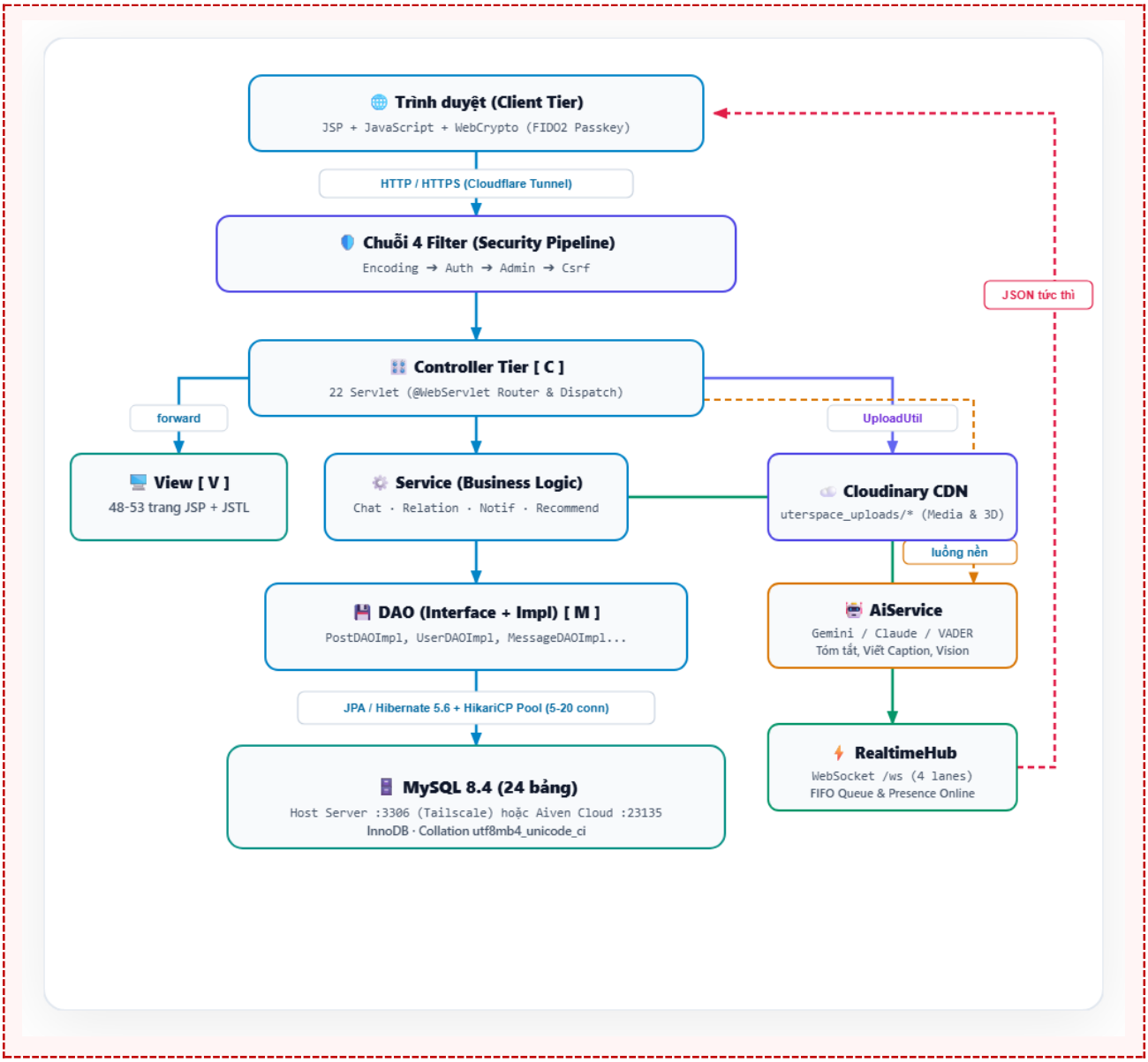
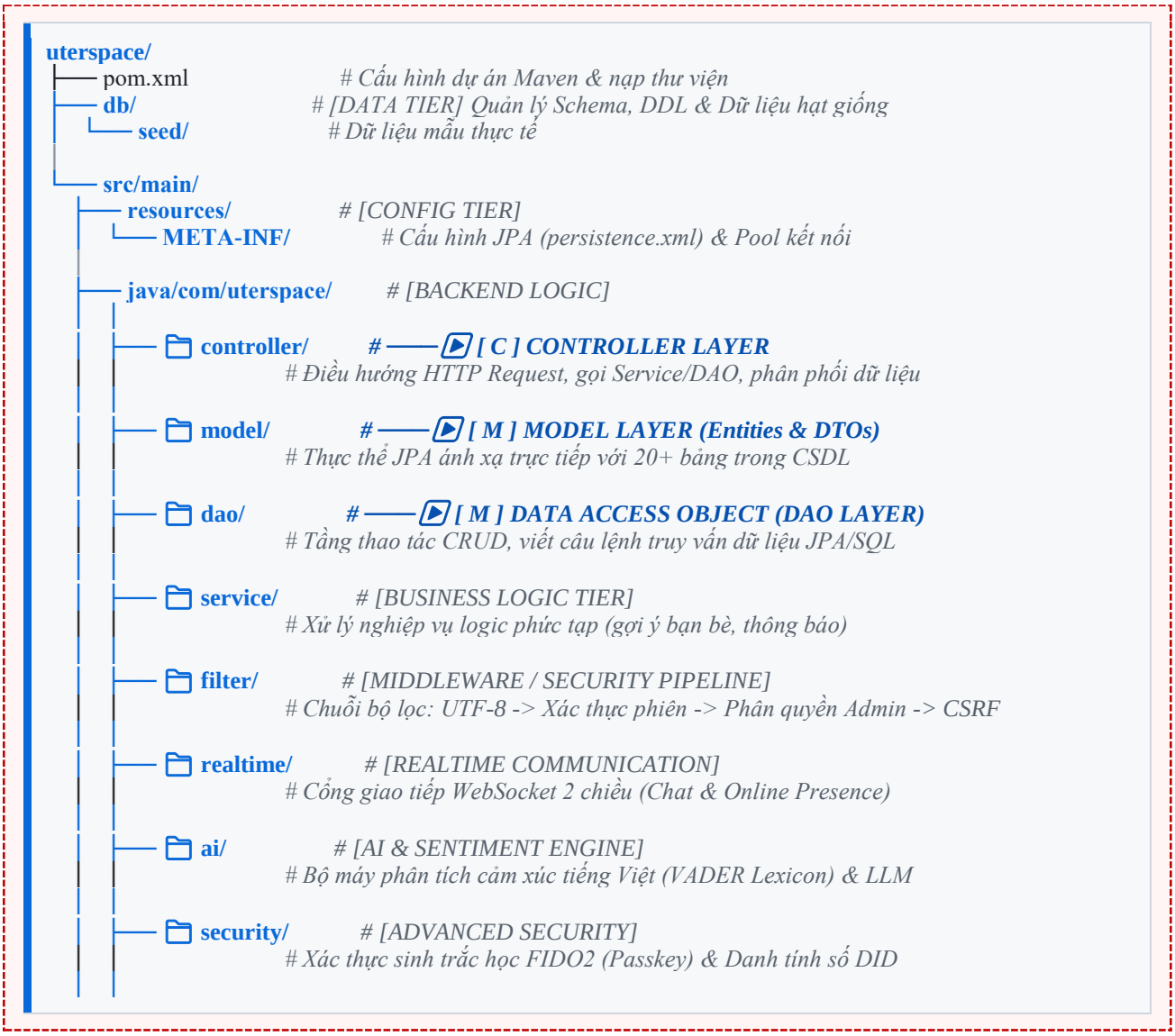


Figure 1.1. Overall architecture diagram of UTErSpace

CHAPTER 2. TECHNOLOGIES USED

2.1. Platform and MVC Architecture

The application is developed with Java 17, packaged into a WAR file using Maven, and runs on Apache Tomcat 9 under Java EE 8 standards, including Servlet 3.1, JSP 2.3, and JSTL 1.2. The codebase is organized following a multi-tier MVC architecture. The View layer comprises 53 JSP pages using only JSTL tags, without any scriptlet Java code. The Controller layer contains 22 Servlets declared using the @WebServlet annotation. Four Service classes—ChatService, RelationService, NotificationService, and RecommendationService—handle shared business logic. The DAO layer cleanly separates interfaces from implementation classes, while the Model layer consists of JPA entities mapped directly to MySQL tables. Thanks to this separation of concerns, database changes only affect the DAO layer without spilling over into the user interface.





2.2. Data Access

The DAO layer uses JPA with Hibernate 5.6 as the object-relational mapping (ORM) framework and HikariCP for connection pooling. The pool maintains a minimum of 5 and a maximum of 20 ready connections, eliminating the overhead of opening a new database connection for each query. Queries are written in JPQL or native SQL and consistently pass parameters through the setParameter method, safeguarding the application against SQL Injection attacks. To minimize query count, comments and like statuses for an entire feed page are loaded in a single batch query rather than per post, eliminating the N+1 query problem, while independent queries such as sidebar counter badges are executed in parallel using the DbParallel helper class.

2.3. User Interface

The user interface is built with semantic HTML using tags like header, nav, main, and article alongside aria attributes for screen reader accessibility, styled with the Poppins font and Heroicons icon set. Vanilla JavaScript consumes APIs via the fetch method and automatically attaches the CSRF token to each request. The newsfeed offers three view modes: three-column grid, two-column grid, and single-post view, with user preferences

saved in localStorage. Clicking a post opens an expanded modal displaying full-sized images or videos along with all comments, while updating the URL hash to #post-id so that the browser's Back button can cleanly dismiss the modal. Additional libraries include Cropper.js for avatar cropping, Pannellum for 360-degree photo viewing, model-viewer for 3D models and augmented reality, an in-browser Whisper model for automatic video captions, and the Web Speech API for text-to-speech post narration.

2.4. Real-time Communication

Real-time features utilize WebSocket adhering to the JSR 356 standard via the /ws endpoint. The RealtimeHub class manages active sessions for each user and delivers events to the correct recipients, including new messages, typing indicators, delivered/read receipts, notifications, and online presence. When an administrator locks an account, the server sends a session termination command, immediately redirecting the locked user's browser back to the login page. In case of WebSocket connection loss, the browser automatically falls back to polling the /api/pulse endpoint periodically so no updates are missed.

2.5. Security

User passwords are encrypted using BCrypt with a work factor of 12. Database passwords and external API keys are kept strictly out of the source code and loaded from Tomcat's external configuration file. All POST requests must include a valid CSRF token, and session cookies are secured with the SameSite=Lax attribute. User-supplied input is escaped upon rendering to prevent Cross-Site Scripting (XSS), and all operations on posts and messages verify ownership or view permissions to prevent Insecure Direct Object References (IDOR).

Password recovery OTP codes are hashed with SHA-256, expire after 5 minutes, and are invalidated after 5 consecutive failed attempts. Uploaded files are verified using binary magic byte signatures rather than relying on file extensions. Sensitive actions, such as changing passwords, unlocking private albums, or linking digital identities, require re-authentication if the user has not confirmed their credentials within the past 5 minutes. Additionally, admin accounts are strictly confined to the /admin area: AuthFilter redirects any access to regular user pages back to the administration section and returns a 403 Forbidden error for non-admin APIs.

2.6. Artificial Intelligence

Post sentiment analysis combines a two-layer approach. The first layer is SentimentAnalyzer, an offline Vietnamese sentiment analyzer built on a weighted lexicon that handles negation, intensifiers, emojis, and unaccented text, normalizing the compound score into the range from -1 to 1 following the VADER methodology. The second layer integrates large language models—Google Gemini or Anthropic Claude—invoked whenever valid API keys are configured on the server. Sentiment analysis runs asynchronously in background threads to avoid delaying post publishing, and if no network connection or API keys are available, the system seamlessly falls back to the offline algorithm. The same foundation powers the AI caption generator from images, post summarization, and translation features.

2.7. Biometric Authentication and Digital Identity

Passkeys adhering to the WebAuthn standard enable biometric authentication via fingerprint or facial recognition directly on user devices. Biometric data never leaves the client device; the server merely stores the public key and verifies ES256 or RS256 signatures, the user verification flag, and a signature counter to prevent replay and cloning attacks. Digital identity follows the W3C DID specification using the did:key method with Ed25519 keypairs. The private key remains securely inside the browser's IndexedDB; users log in by signing a one-time cryptographic challenge, and can export their student profile as a Verifiable Credential signed using the eddsa-jcs-2022 cryptosuite for independent third-party integrity verification.

2.8. Media Storage on Cloudinary

Avatars, cover photos, post images and videos, stories, and 3D models are stored on Cloudinary, while the database retains only their corresponding paths. Files uploaded by users reside in the uterspace_uploads folder, and seed data files are organized under uterspace_seed. When serving media, the MediaUrl helper transforms paths into /cdn/... endpoints, and CloudMediaController issues a 301 redirect to Cloudinary's CDN servers for efficient browser caching. When a post is deleted or an avatar is updated, a background thread automatically cleans up the orphaned file on Cloudinary to conserve storage.

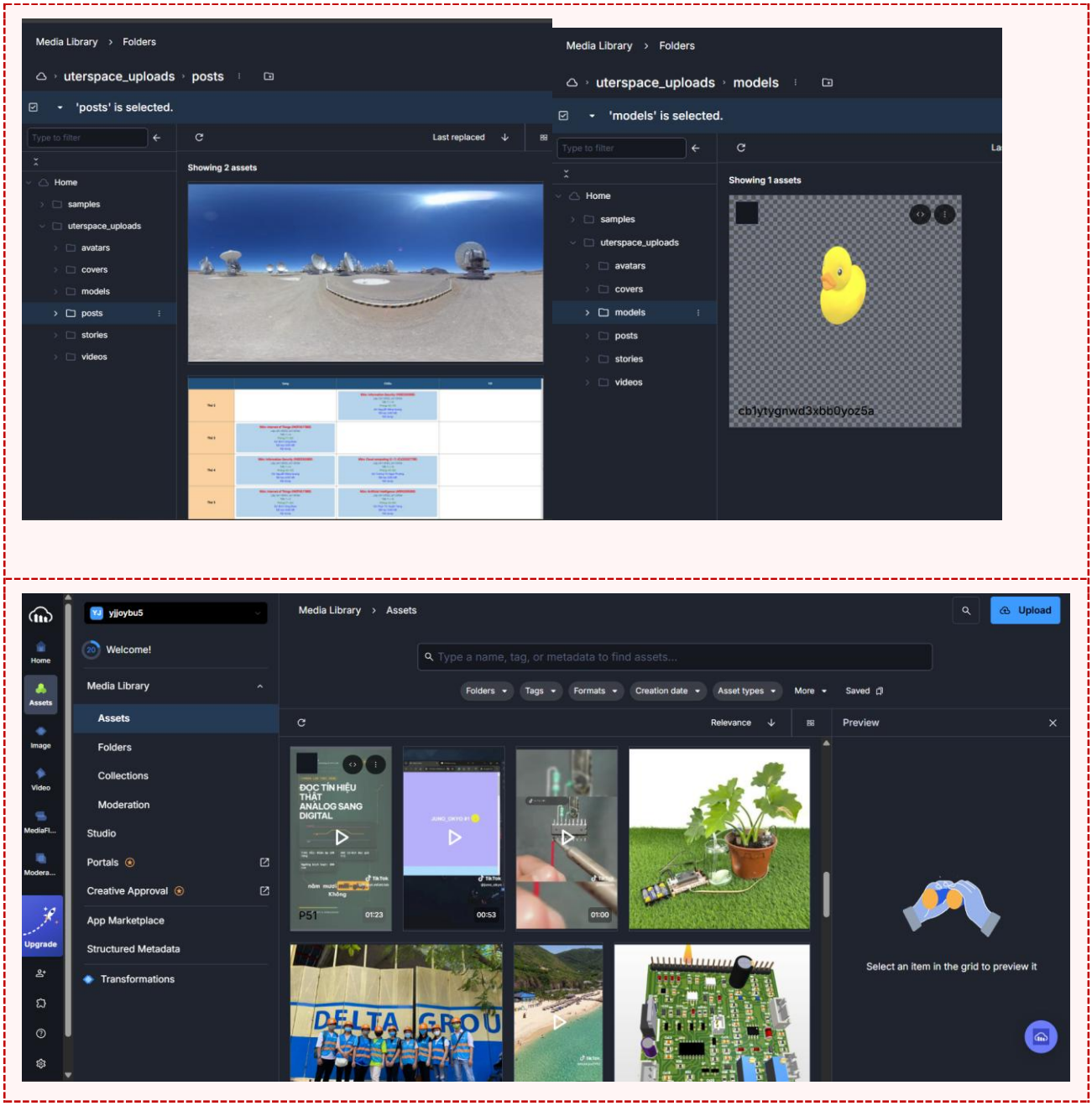


Figure 2.1. UTErSpace media library on Cloudinary

2.9. Deployment Infrastructure

The MySQL 8.4 database is hosted on the team's server and shared securely through a Tailscale Virtual Private Network (VPN), allowing all team members to work against the exact same dataset with only a few milliseconds of latency. Source code is version-controlled on GitHub, with each member developing on dedicated feature branches. For project presentations, the team employs Cloudflare Tunnel to expose a public HTTPS endpoint routing to Tomcat. HTTPS is a mandatory requirement for WebAuthn Passkeys, Ed25519 cryptographic APIs, and secure WebSockets to function across mobile phones and external laptops. Repetitive tasks such as spinning up the MySQL server, executing

SQL migrations, database backup and restoration, restarting Tomcat, and toggling Cloudflare tunnels are fully automated using PowerShell scripts.

CHAPTER 3. SYSTEM FEATURES

3.1. Registration, Login, and Password Recovery

Users register with an email address and a password of at least 8 characters containing uppercase letters and special characters, while agreeing to the terms of service. Authentication is supported via two methods: email and password, or passwordless DID digital identity login. In case of a forgotten password, users provide their registered email, receive a verification OTP code via Gmail, and set a new password. AuthController and ForgotPasswordController process these requests and manage state in the users table.

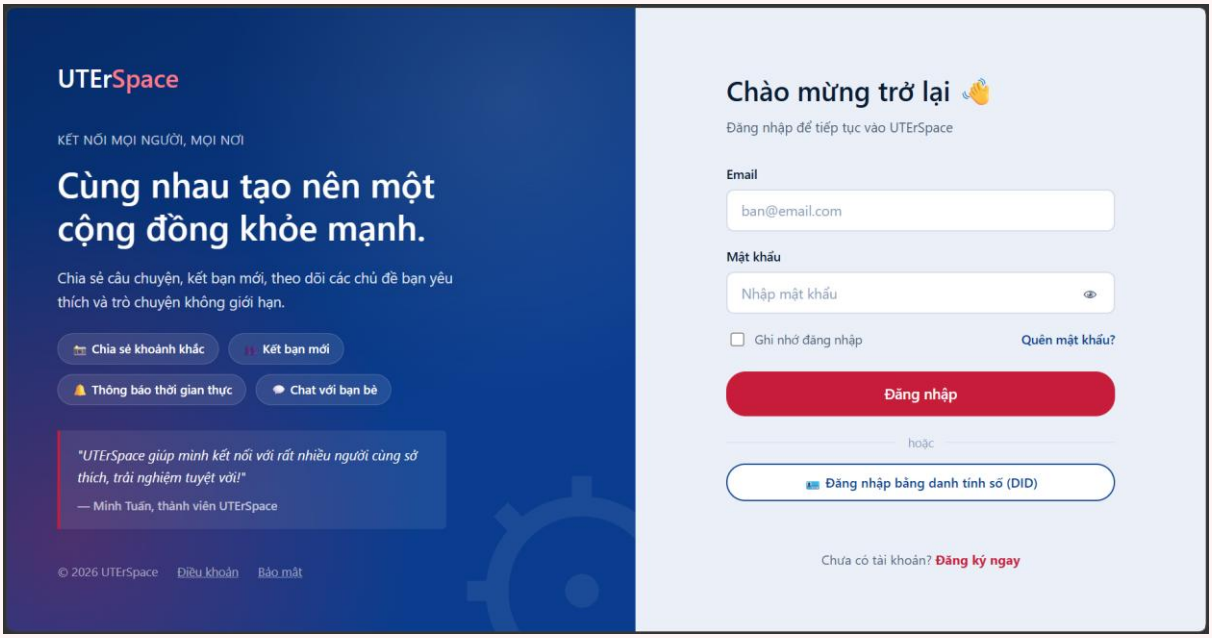


Figure 3.1. Login page

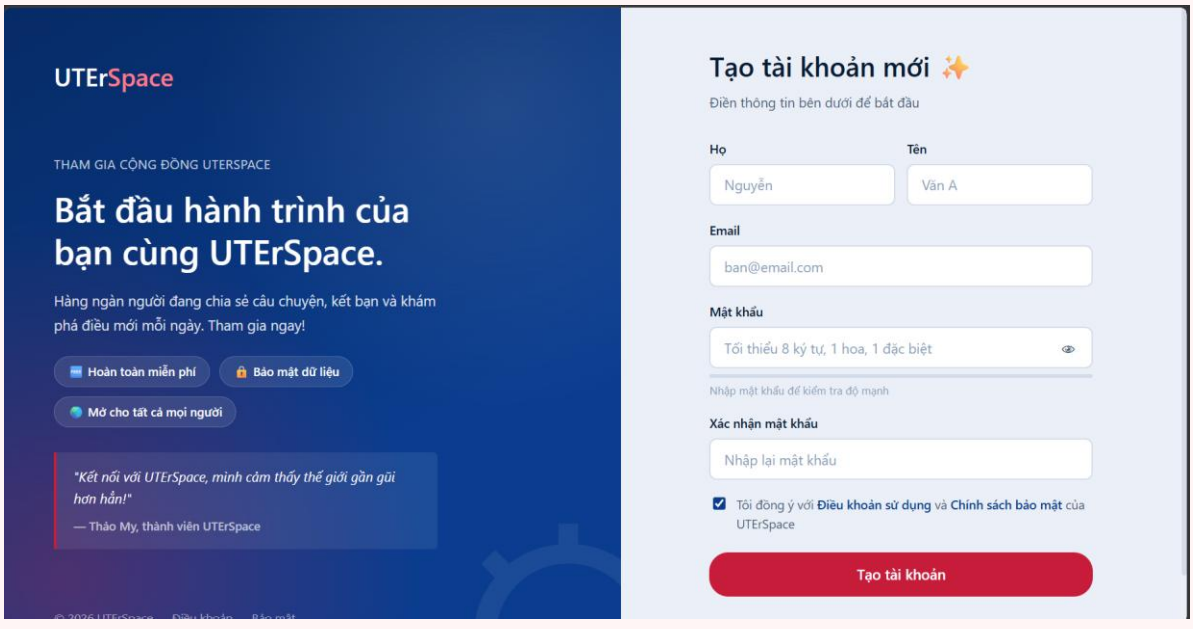


Figure 3.2. Account registration page

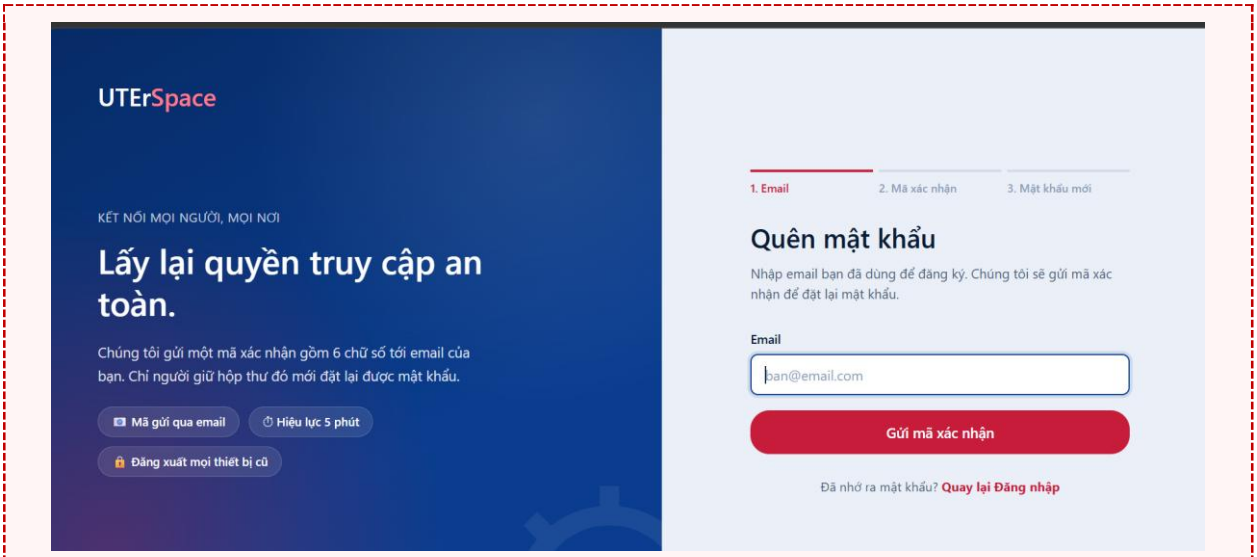


Figure 3.3. Password recovery using OTP

3.2. Newsfeed

The newsfeed serves as the primary home page upon logging in. Users can create posts containing text, images, videos, 360-degree photos, depth-map 3D photos, or 3D models, with configurable visibility scopes: Public, Friends, or Only Me. Each post supports liking, commenting, bookmarking, editing, deleting, hiding, or quote-sharing with additional commentary, where the original post is rendered embedded inside the new post. Hashtags within post captions automatically transform into clickable links routing to hashtag-filtered views.

The 'For You' feed mode ranks posts according to an interest score calculated from engagement levels, preferred topics, and trending topics, complete with an explanation badge for why each post is recommended; the 'Latest' mode orders posts chronologically. The feed can be toggled between three-column grid, two-column grid, and single-post views, with each post featuring an AI-assigned sentiment tag. The processing flow traverses FeedController, RecommendationService, and corresponding DAOs for posts, likes, and comments, interacting with the posts, likes, comments, and saved_posts tables.

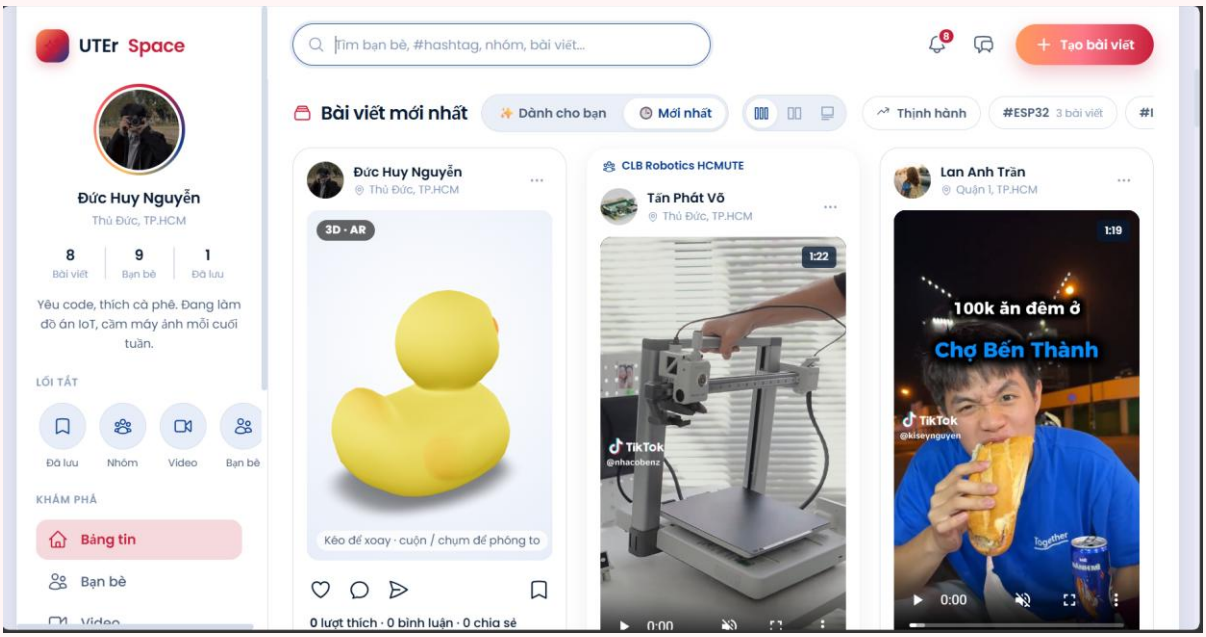


Figure 3.4. Three-column grid newsfeed

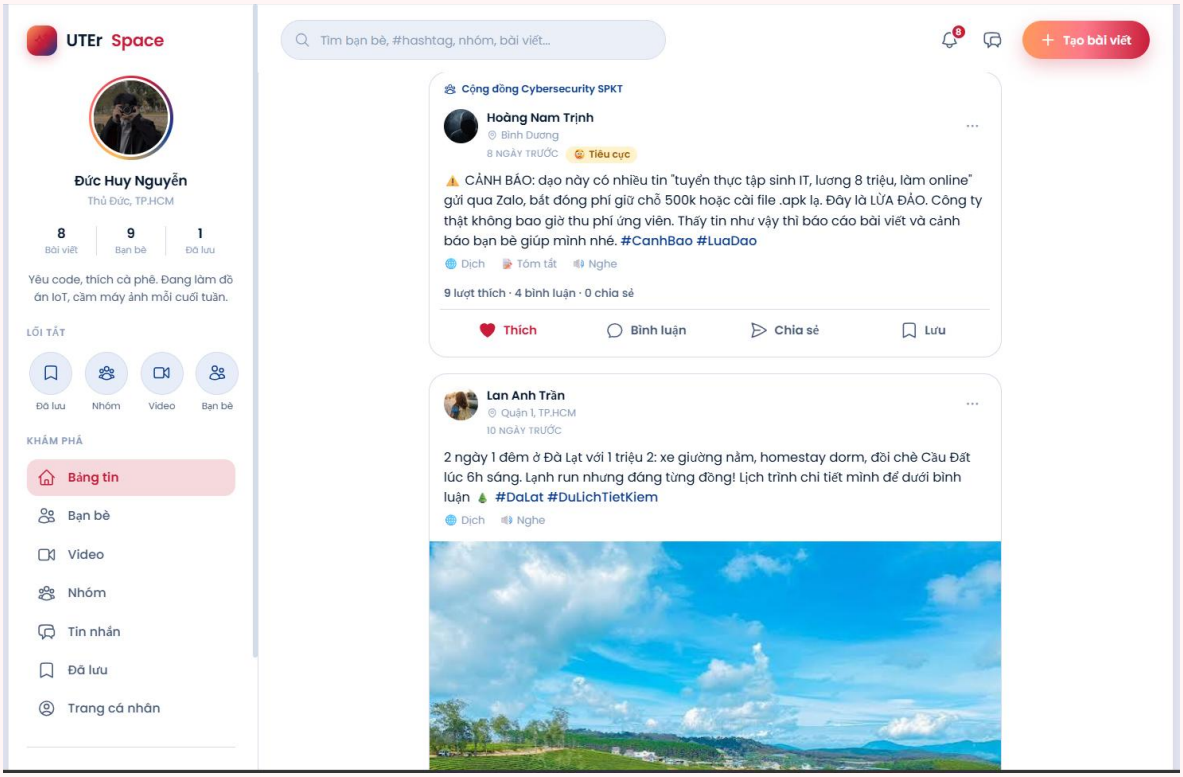


Figure 3.5. Single-post feed view

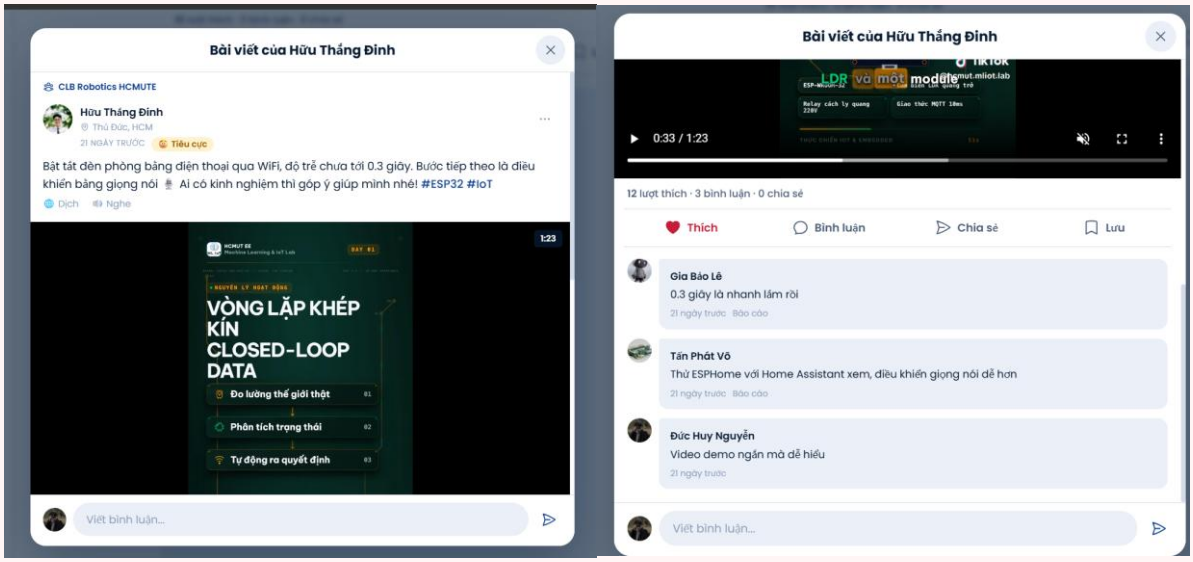


Figure 3.6. Post detail and comments modal

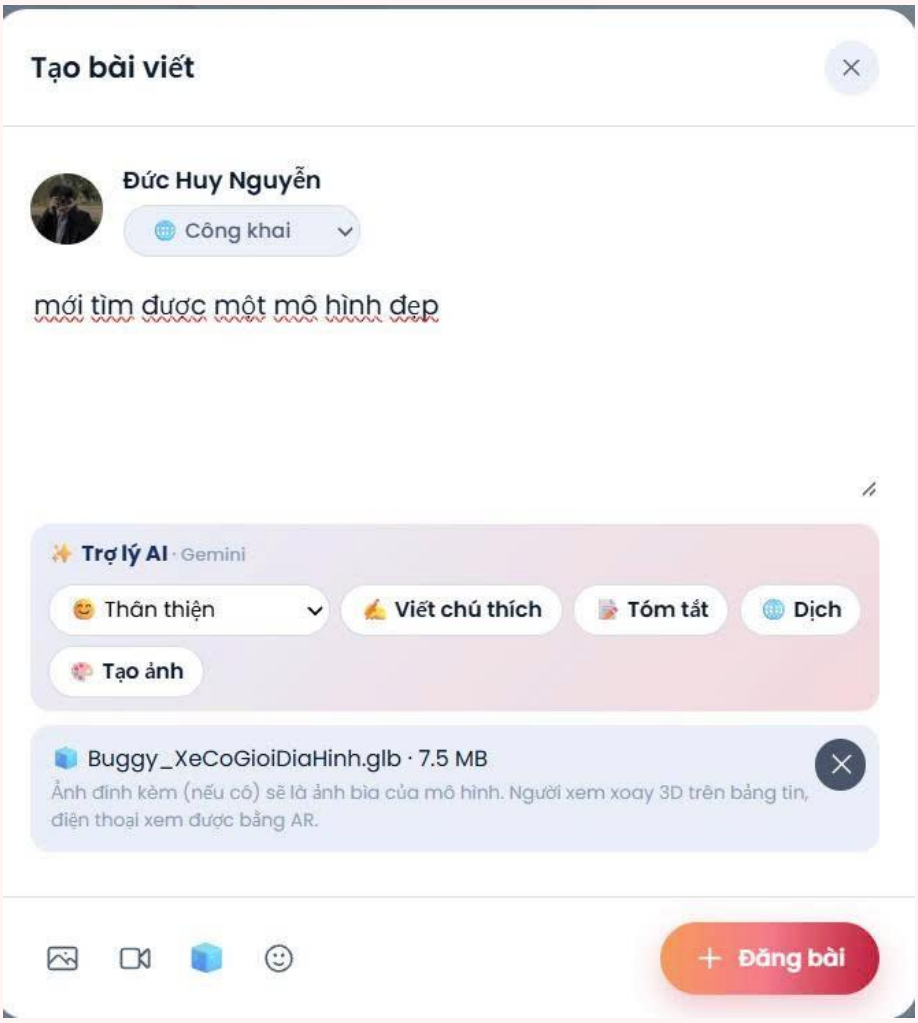


Figure 3.7. New post creation box

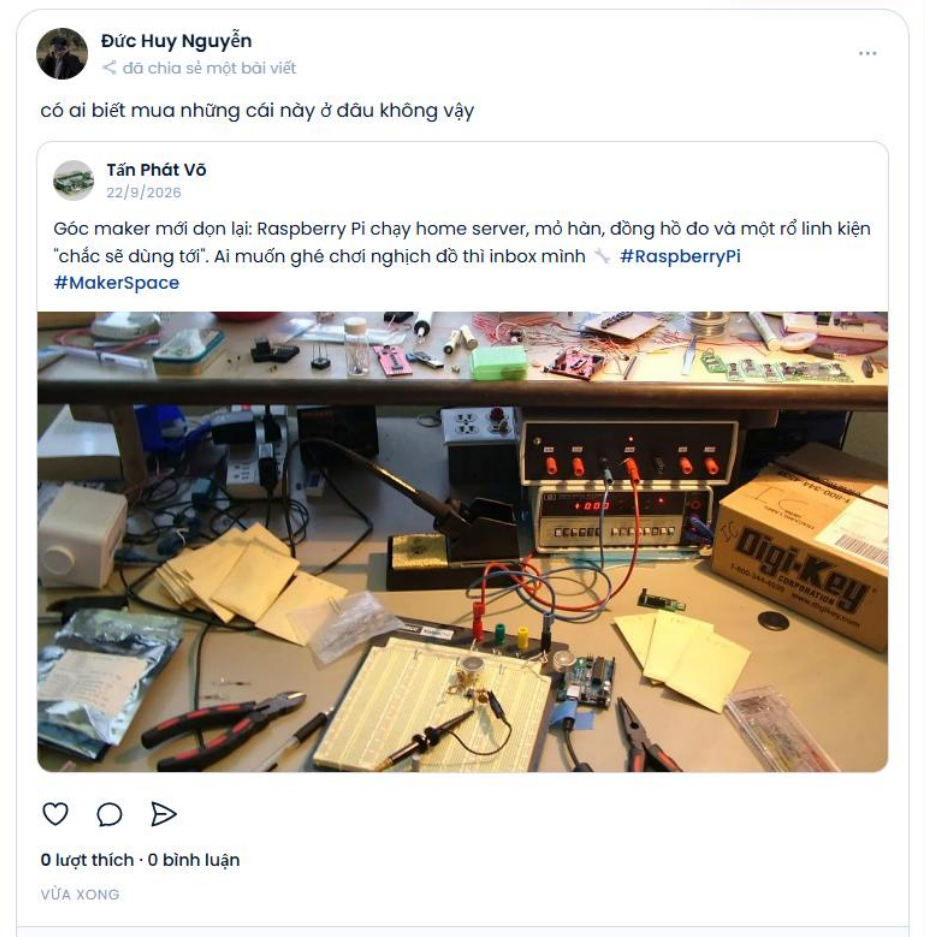


Figure 3.8. Shared post with quote commentary

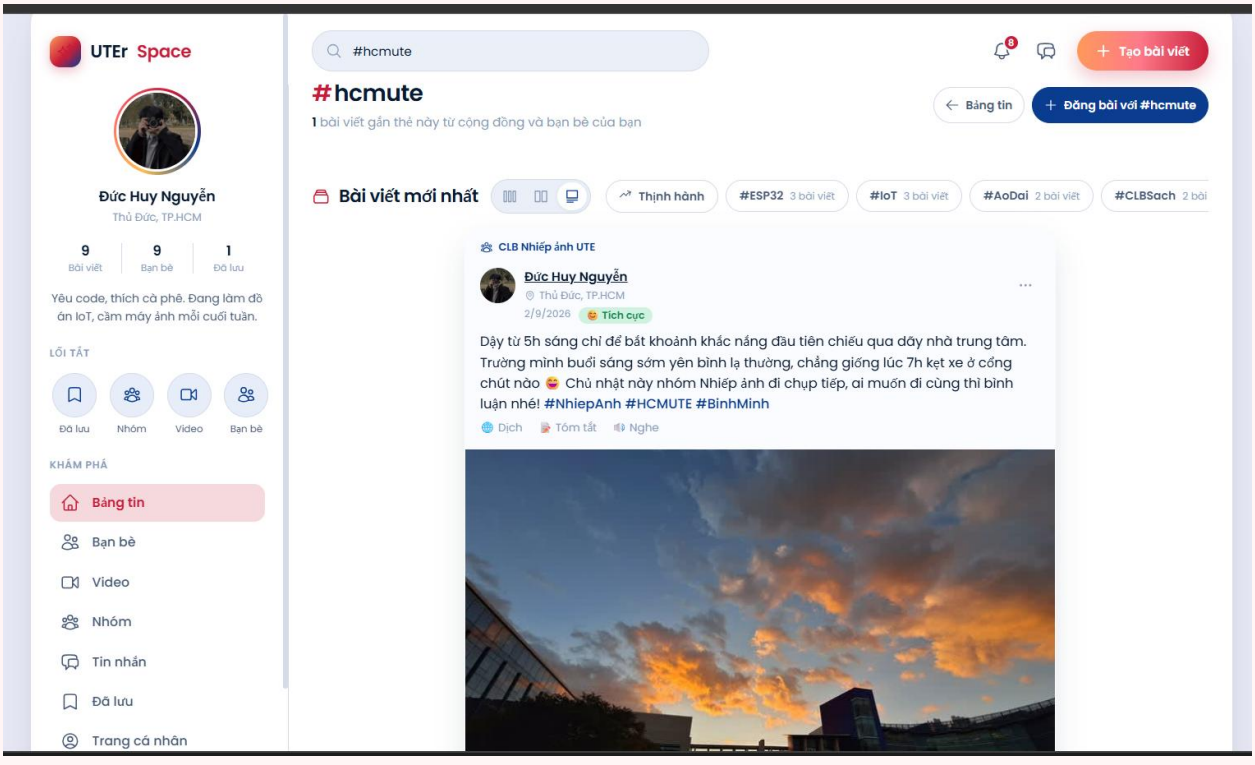


Figure 3.9. Hashtag filter page

3.3. Stories

Stories are photo or video snippets that persist for 24 hours, displayed as an avatar carousel along the top of the newsfeed. Friends can view, react with hearts, or reply to a story, with replies automatically forwarded as private direct messages. Story owners can inspect the viewers list, where users who reacted with hearts are sorted to the top. Expired stories are automatically archived in a private archive if the user enables this option. StoryController and StoryDAOImpl handle this workflow across the stories, story_views, and story_reactions tables.

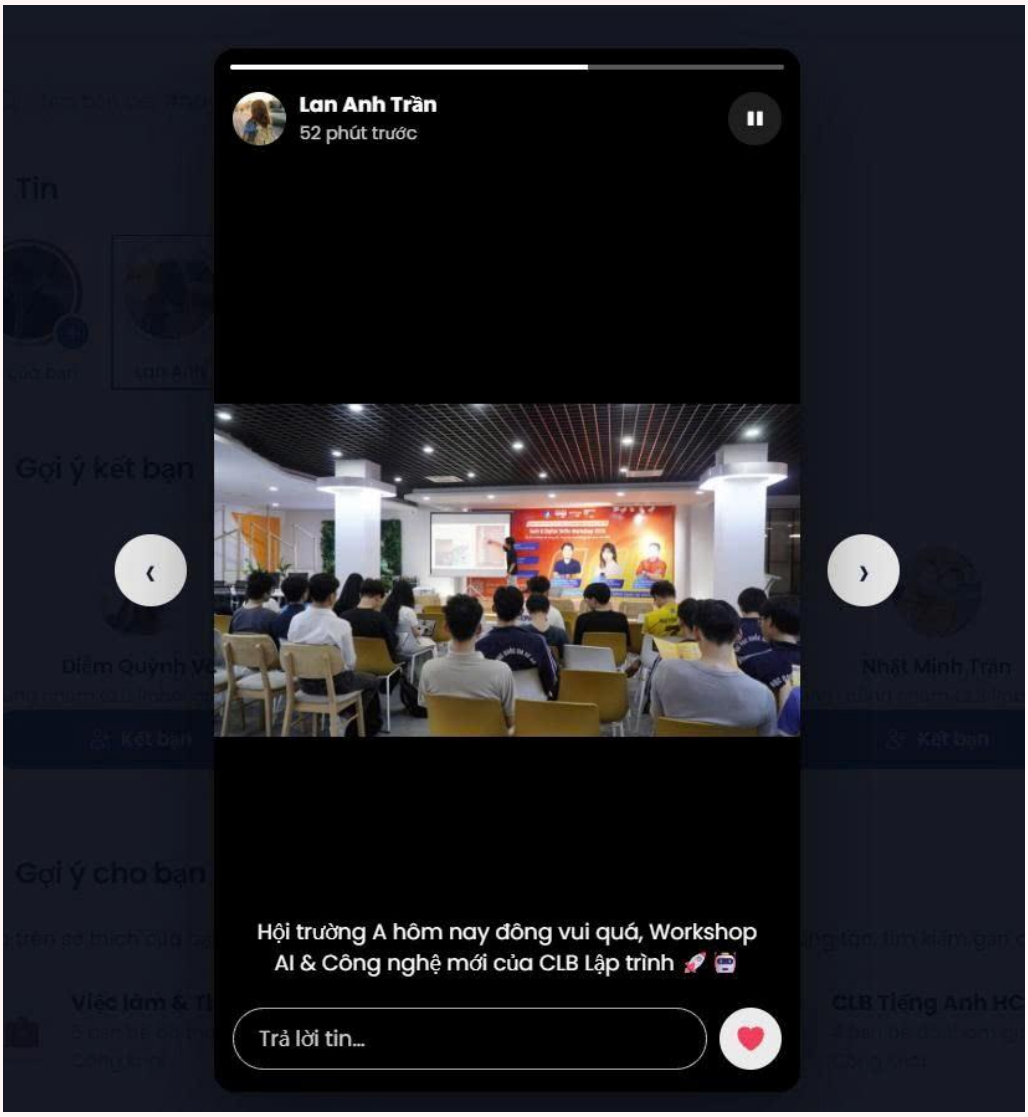


Figure 3.10. Story viewer

3.4. Profile Page

The profile page displays the user's avatar, cover banner, bio summary, personal posts, media gallery, and friend count. Avatars and cover photos can be cropped

interactively in the browser prior to upload. Accounts that have verified and linked a digital identity display a verified badge next to their username.

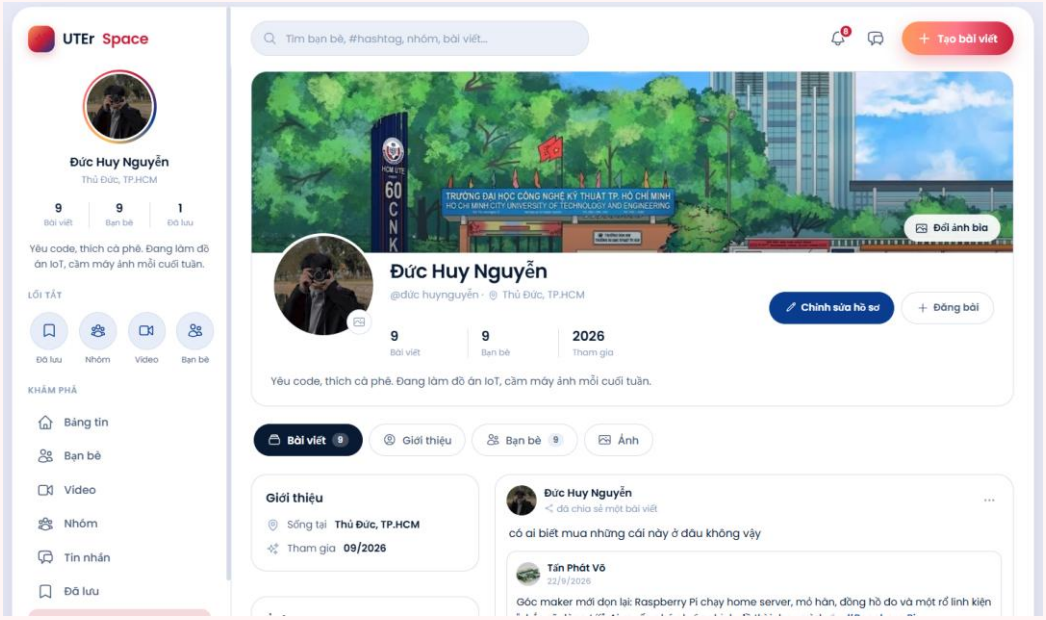


Figure 3.11. User profile page

3.5. Friends

The Friends page consists of four tabs: friend list, received requests, sent requests, and friend suggestions. Each suggestion includes mutual context such as mutual friend count, shared groups, or overlapping interests. Users can block or unblock others at any time. Between any pair of users, only a single unique relationship record exists in the friendships table, even if both users send requests simultaneously. The request pipeline involves FriendsController, RelationService, and FriendshipDAOImpl.

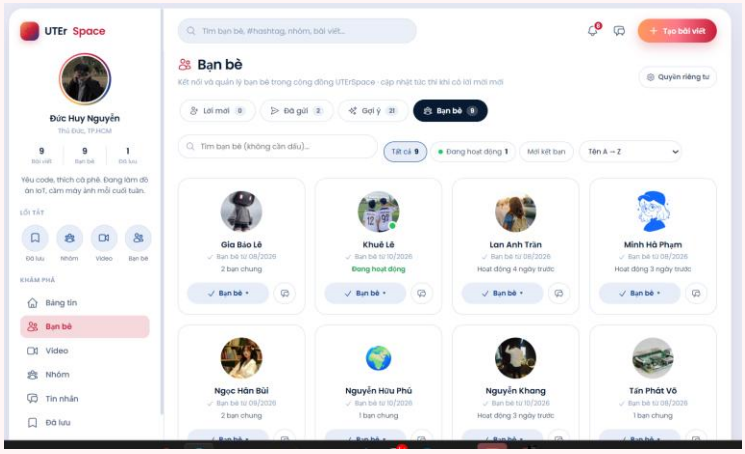


Figure 3.12. Friends page

3.6. Messaging

The messaging feature supports one-on-one direct messages and group chats, with real-time status indicators for typing, delivered, and read receipts. Users can react with emojis, reply in threads, pin, edit, recall, delete messages locally for themselves, send images and file attachments, and search messages without diacritics. Consecutive messages from the same sender sent within 5 minutes are grouped into a single chat bubble cluster, while messages separated by 15 minutes or more display a timestamp divider, keeping conversation threads clean and readable. ChatController and ChatService manage this pipeline across the conversations, conversation_members, chat_messages, message_reactions, and message_hidden tables, broadcasting events via WebSocket.

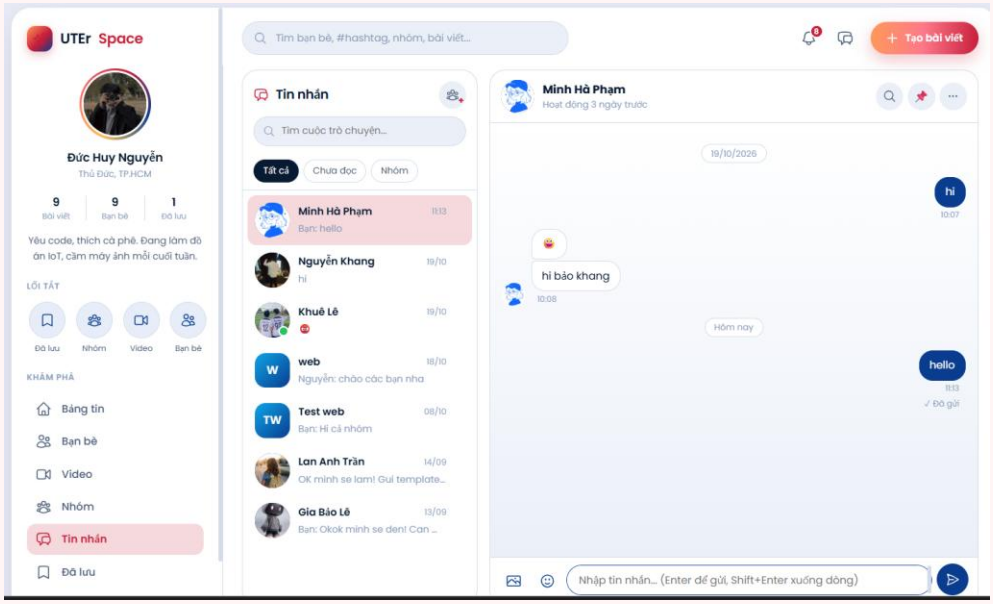


Figure 3.13. Chat and messaging interface

3.7. Notifications

The system generates instant notifications for likes, comments, shares, friend requests, group activities, incoming messages, or story reactions, delivering them immediately to recipients over WebSocket. Notifications referring to the same target entity are aggregated to prevent notification spam. Users can filter notifications by category, mark them as read, or delete them.

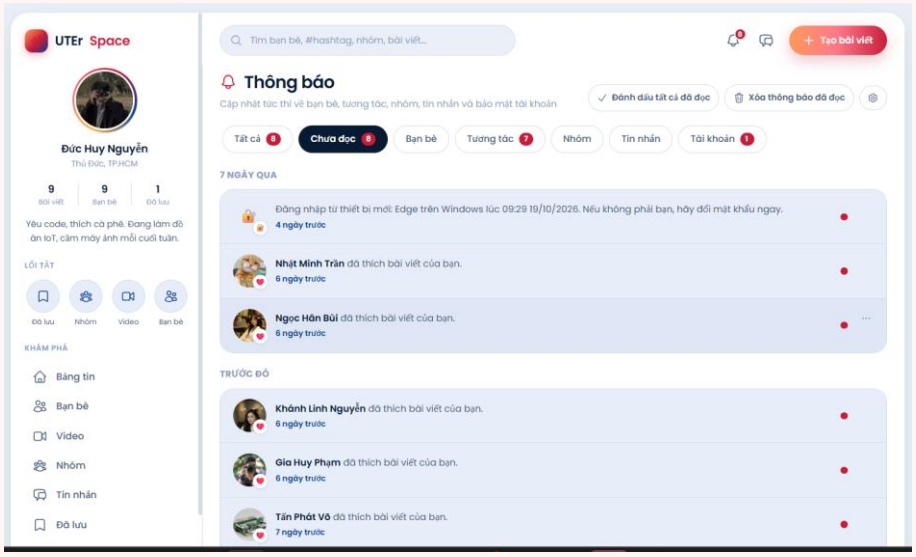


Figure 3.14. Notifications page

3.8. Groups

Users can create public or private groups; private groups require group owner approval for new members, and only confirmed members can view posts within the group. Group members hold roles of owner, admin, or regular member, and the owner can transfer ownership to another member. GroupsController verifies permissions via the GroupPermission utility class before persisting changes into the groups_tbl and group_members tables.

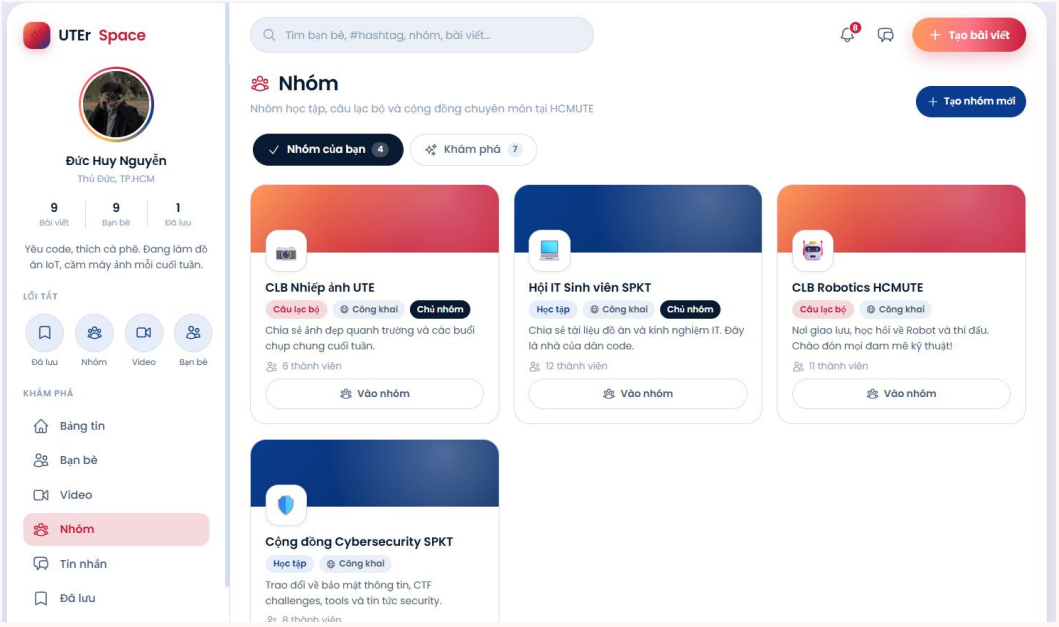


Figure 3.15. Groups list

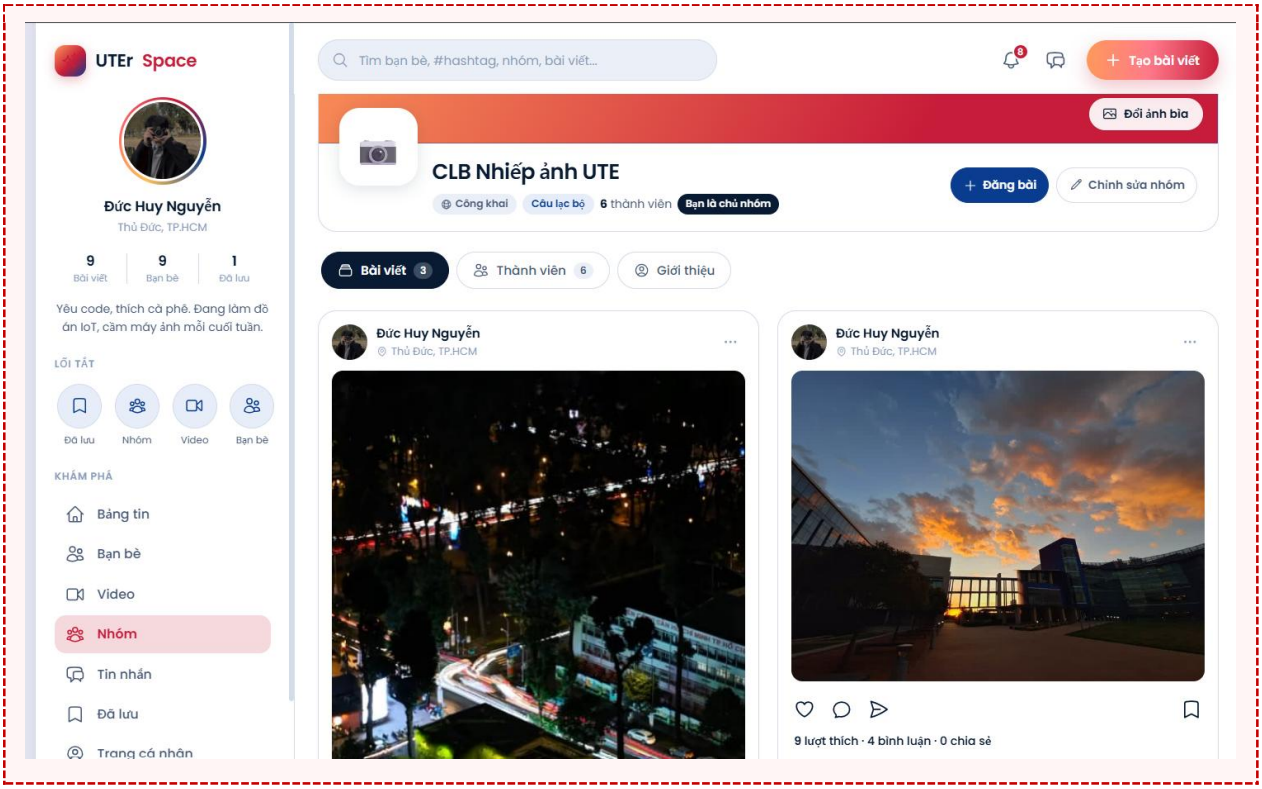


Figure 3.16. Group detail page

3.9. Videos and Saved Posts

The Video page aggregates all posts containing video content. Each video automatically scales to fit the viewport, featuring automated closed captions and a text-to-speech voiceover that narrates the subtitles aloud. The Saved page displays all posts that the user has bookmarked for later reading.

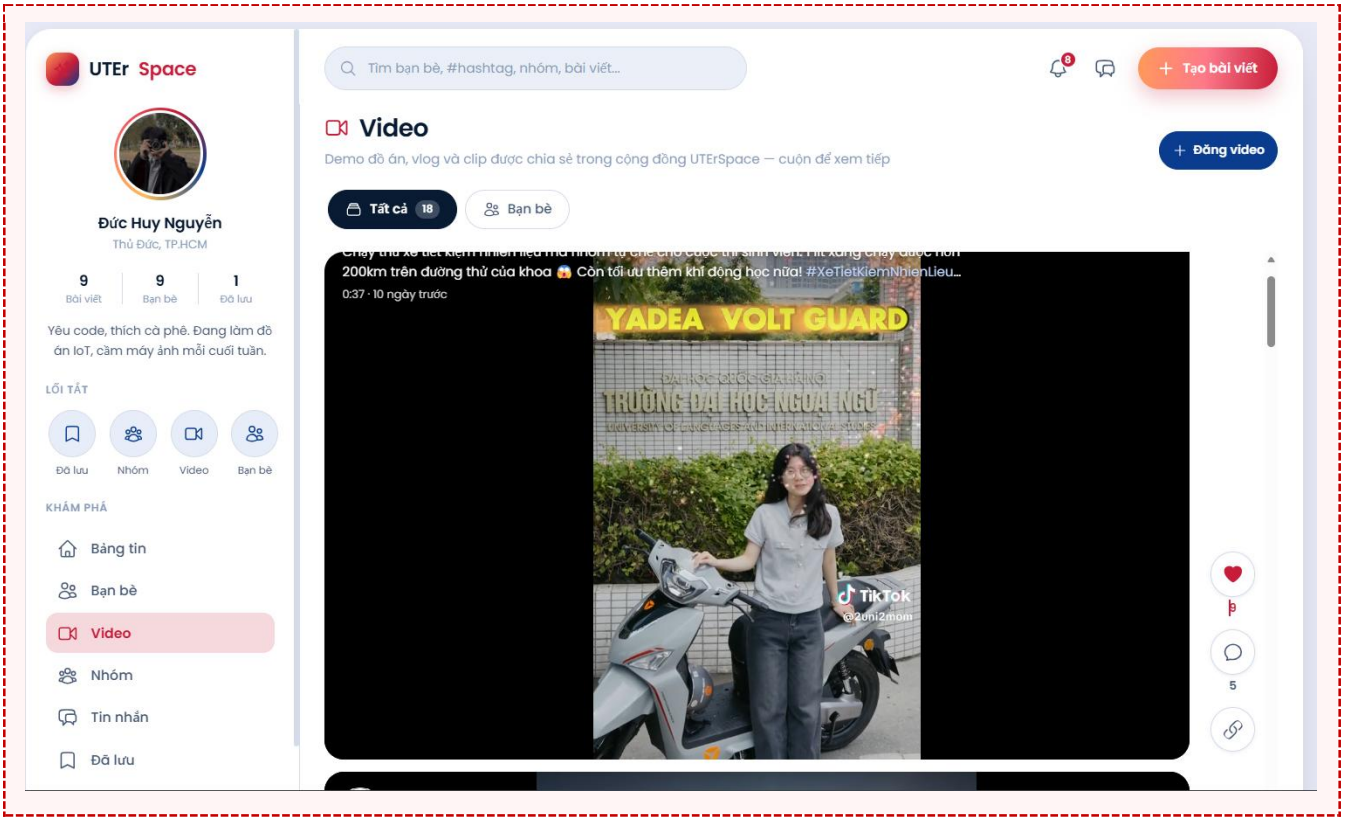


Figure 3.17. Video page

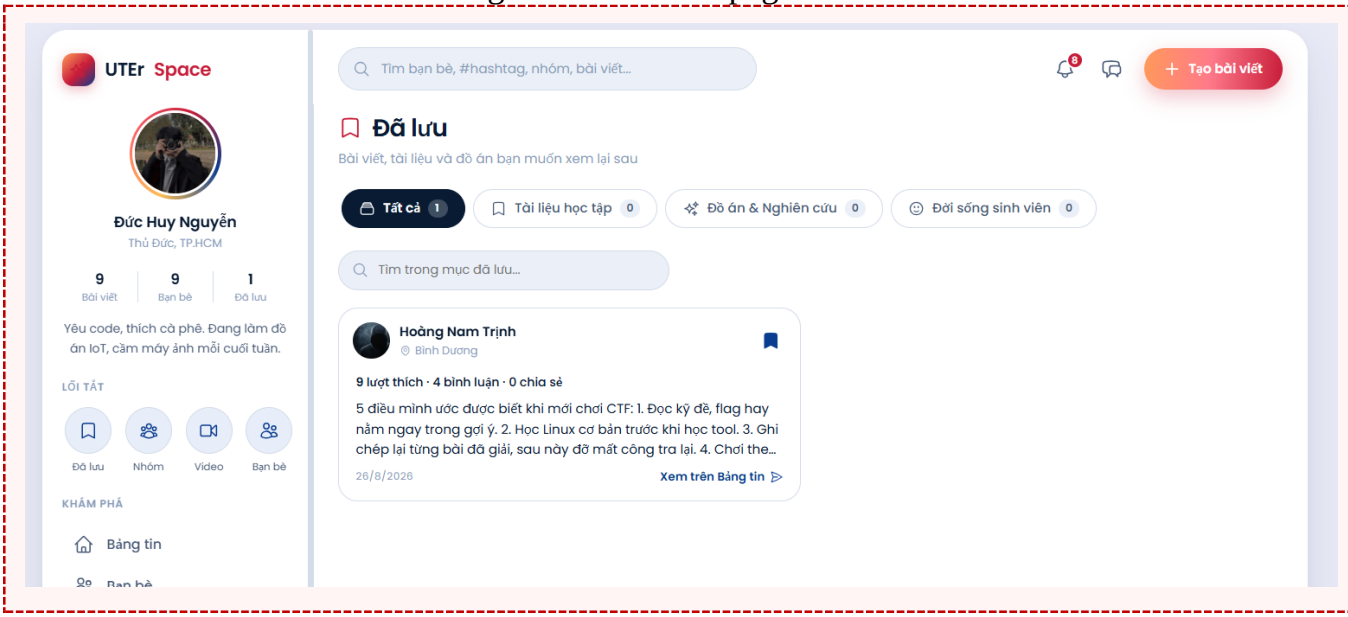


Figure 3.18. Saved posts page

3.10. Accent-Insensitive Search

The search bar on the top navigation bar supports accent-insensitive searching: typing 'duc' matches 'Đức', and typing 'do an' matches 'Đồ án'. As the user types, the system offers instant live suggestions for users, groups, and hashtags; the full results page further discovers matching posts and highlights search keywords in yellow. Post discovery operates in two steps: a native SQL query uses the utf8mb4_0900_ai_ci collation for case-

and accent-insensitive text matching, and the results are subsequently filtered through general privacy visibility rules, ensuring private posts are never leaked via search results. SearchController and SearchDAO execute this workflow, logging queries into the search_history table to power future recommendations.

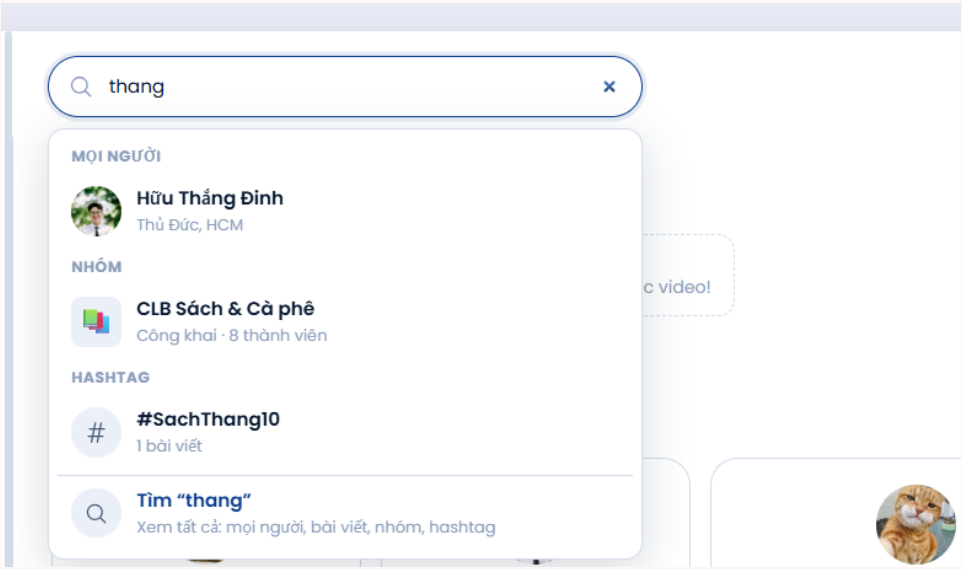


Figure 3.19. Instant search suggestions while typing

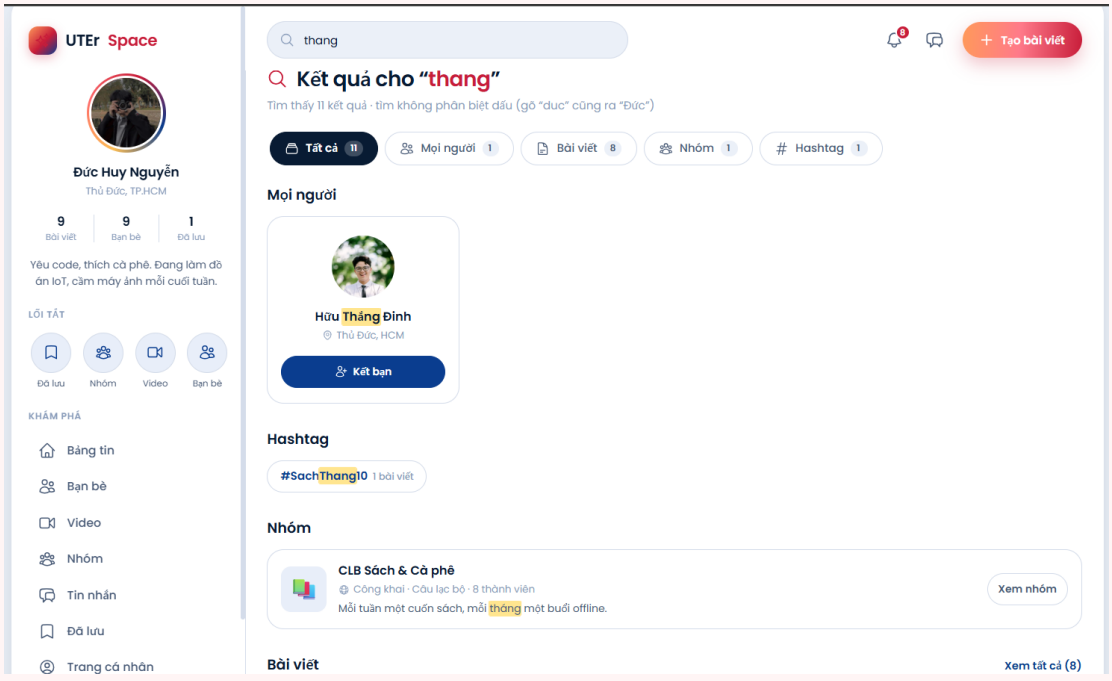


Figure 3.20. Search results page

3.11. AI Assistant

Within the post creation box, an AI assistant crafts captions tailored to a user-selected tone and can incorporate context from attached images. On every published post, action buttons allow translating the text into six languages, generating a quick summary of lengthy

content, or listening to text-to-speech post narration. When a user is about to publish content with strongly negative sentiment, the system surfaces a gentle, supportive prompt prior to submission, while administrators can filter posts flagged with negative sentiment labels. The processing pipeline connects AiController, AiService, SentimentAnalyzer, and Gemini or Claude integration clients, storing sentiment evaluations in the sentiment columns of the posts table.

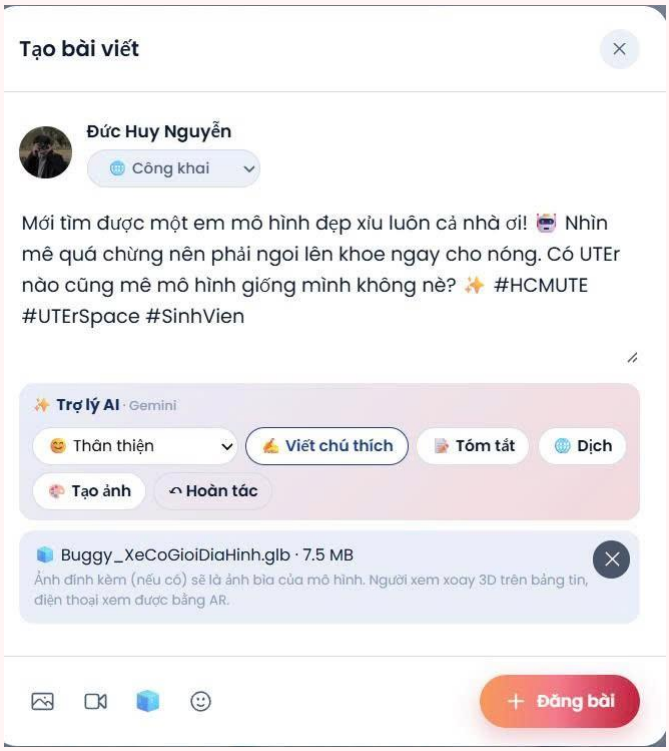


Figure 3.21. AI assistant generating captions

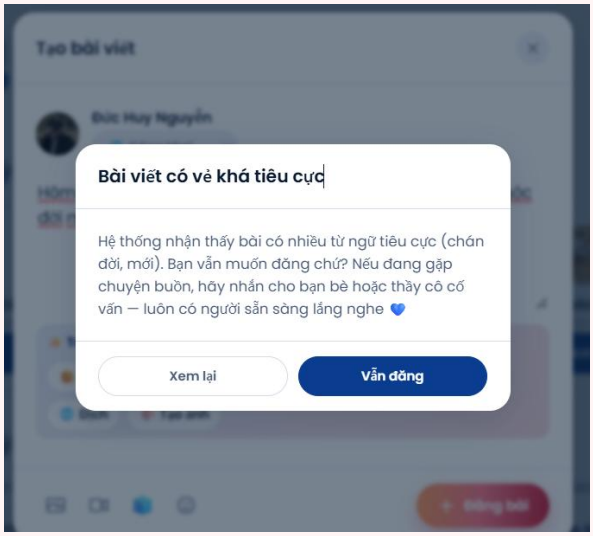


Figure 3.22. Gentle reminder when posting negative content

3.12. Settings

The Settings page includes eight sections: account, privacy, notifications, feed and content filter, story archive, security, digital identity, and blocked users list. The personal content filter allows users to define keywords or emojis they wish to avoid, automatically collapsing posts containing them on their newsfeed. All preferences are persisted in the user_settings table.

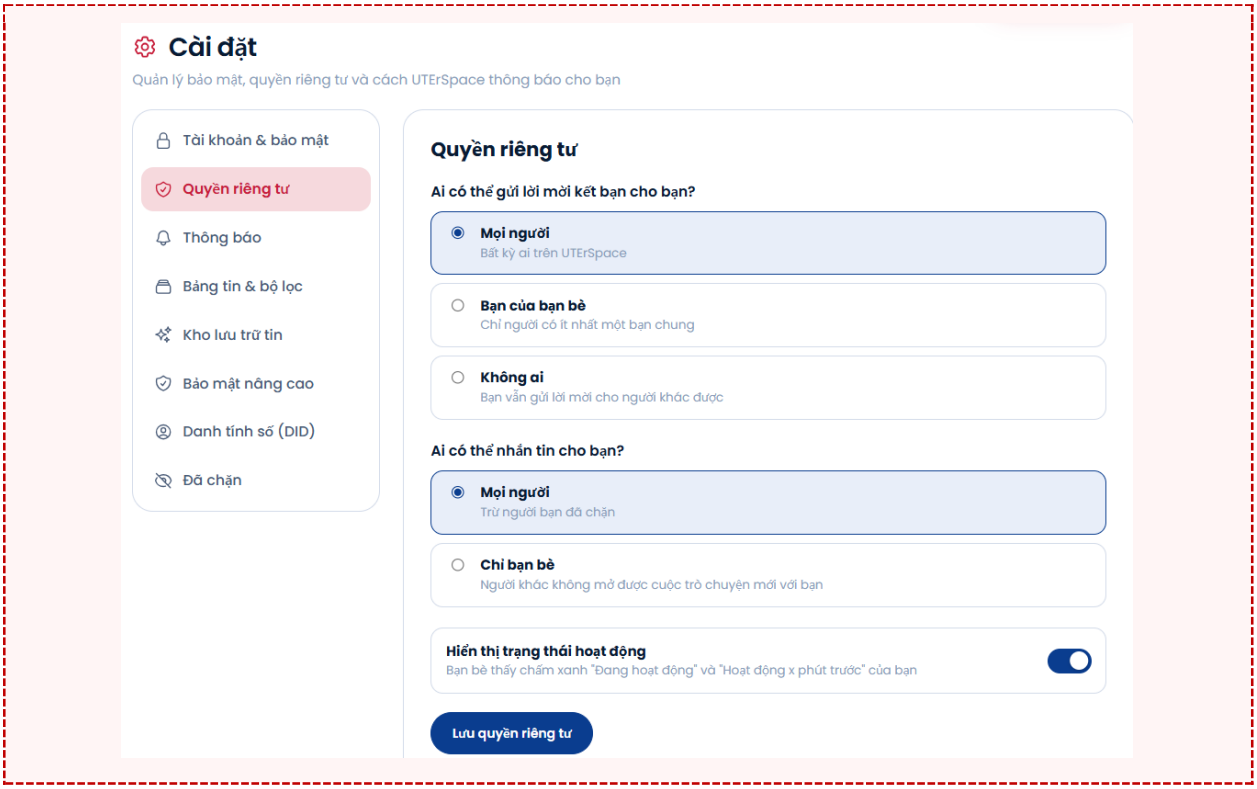


Figure 3.23. Privacy settings

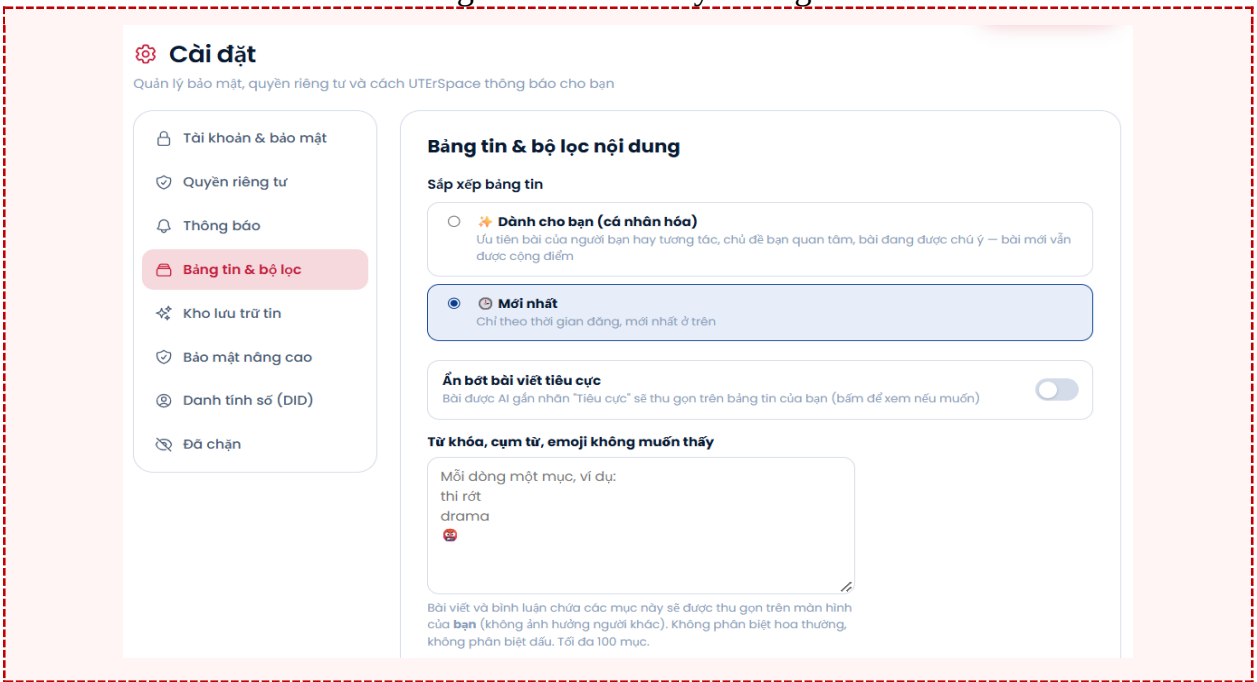


Figure 3.24. Personal content filter

3.13. Passkeys and Digital Identity

Users can register a Passkey using their fingerprint or facial recognition, subsequently using it to unlock hidden albums and authenticate sensitive actions. Under the Digital Identity section, users generate a did:key cryptographic keypair, back up their private key, link the identity to their account, and export their student profile as a signed Verifiable Credential file. The public /did/verify page allows anyone to drag and drop this credential file to verify cryptographic signature validity. SecurityController and DidLoginController orchestrate these capabilities, storing credentials in the webauthn_credentials and user_dids tables.

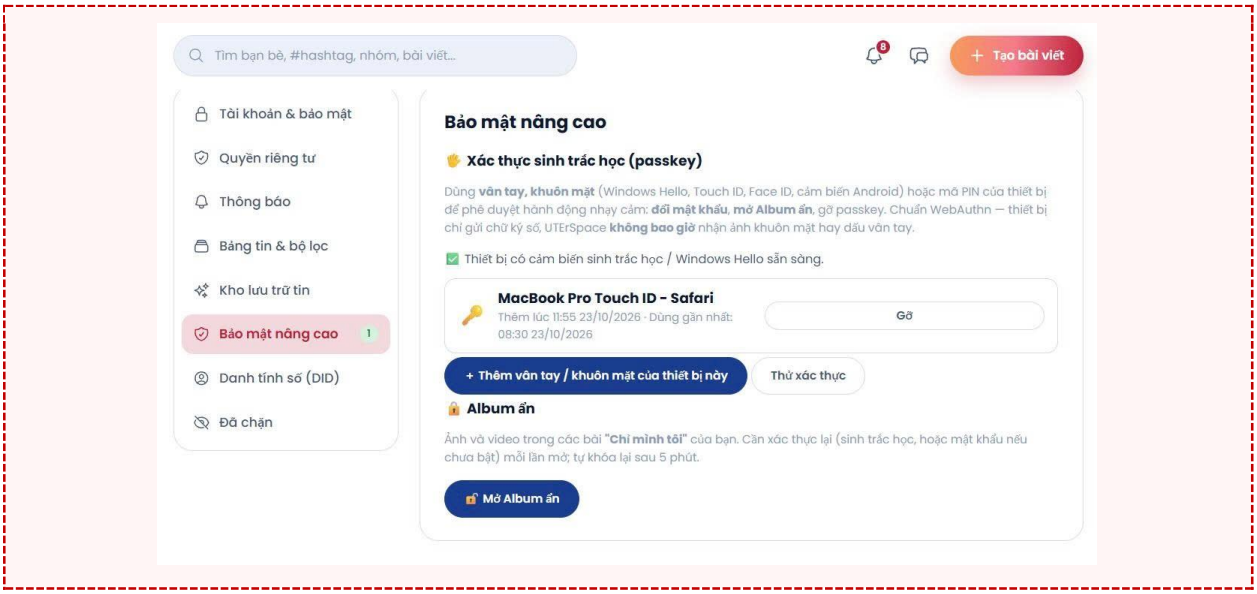


Figure 3.25. Passkey management

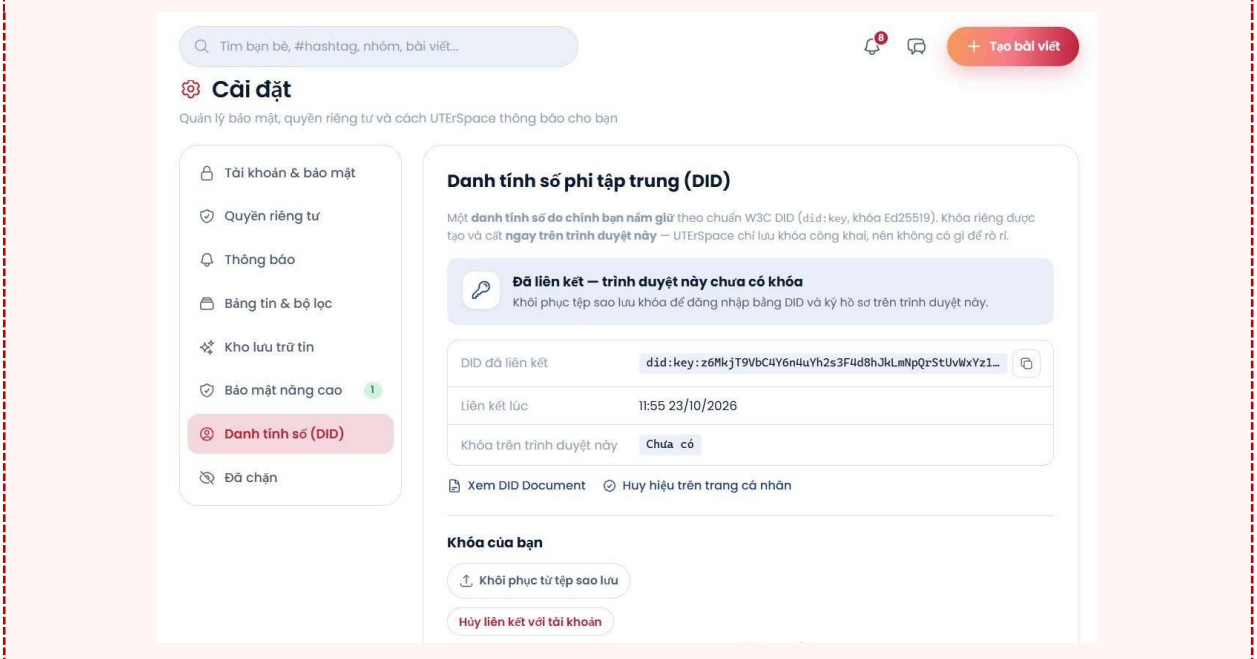


Figure 3.26. Linked digital identities

3.14. Report Violations

Users can report offending posts, comments, or user accounts by submitting a violation reason. Each user can report a specific target entity only once, and multiple reports regarding the same entity are consolidated so that administrators can resolve them in a single review workflow.

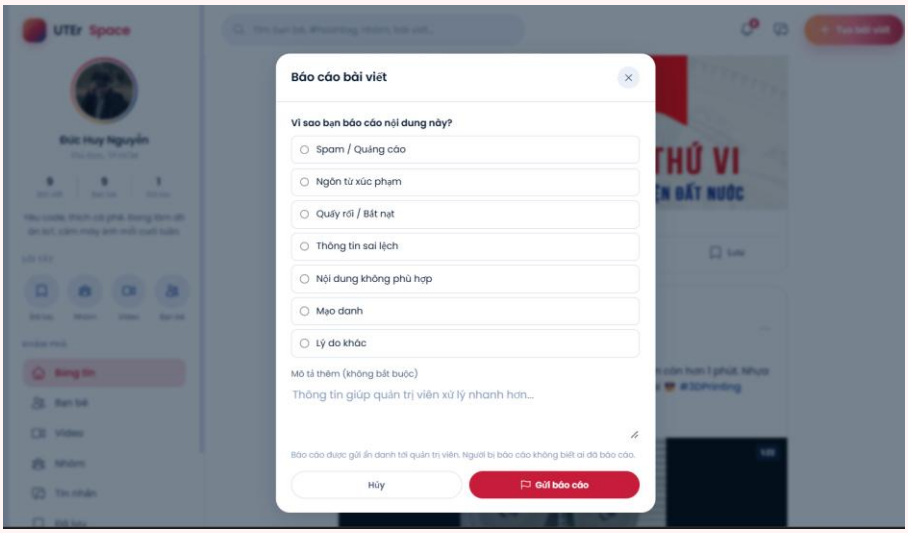


Figure 3.27. Report violation dialog

3.15. Administration

The Administration area is restricted exclusively to ADMIN accounts. The dashboard presents platform metrics, a seven-day activity chart, and a timeline of recent system events. Administrators can suspend or activate user accounts, resolve violation reports, hide policy-violating posts, manage groups, and inspect comprehensive audit logs of all administrative actions stamped with timestamps and IP addresses. The admin interface supports both light and dark themes. AdminController operates protected behind AdminFilter, logging events into the reports and admin_logs tables.

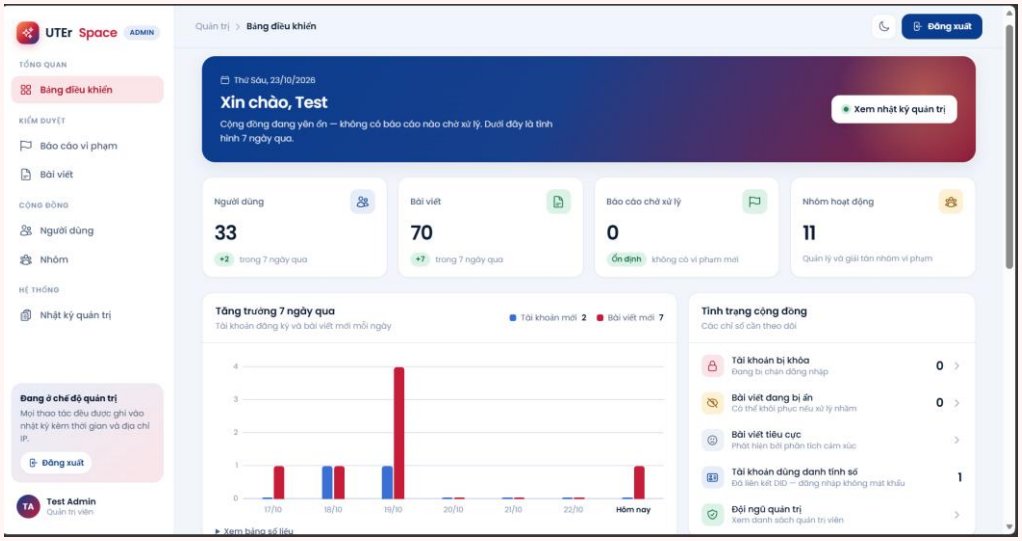


Figure 3.28. Admin dashboard overview

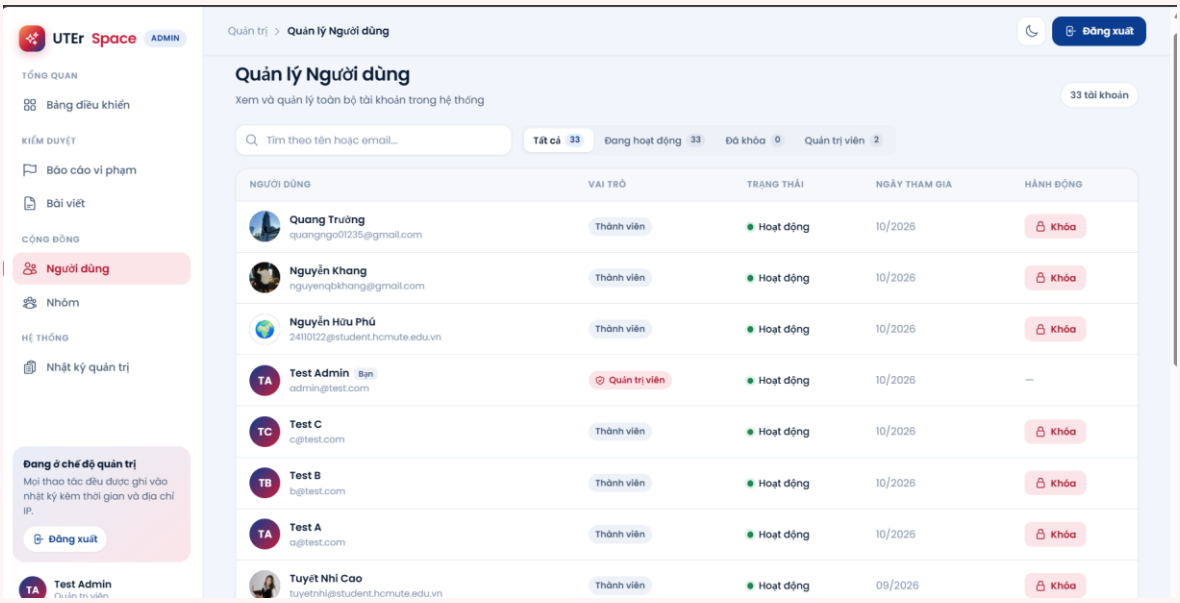


Figure 3.29. User management

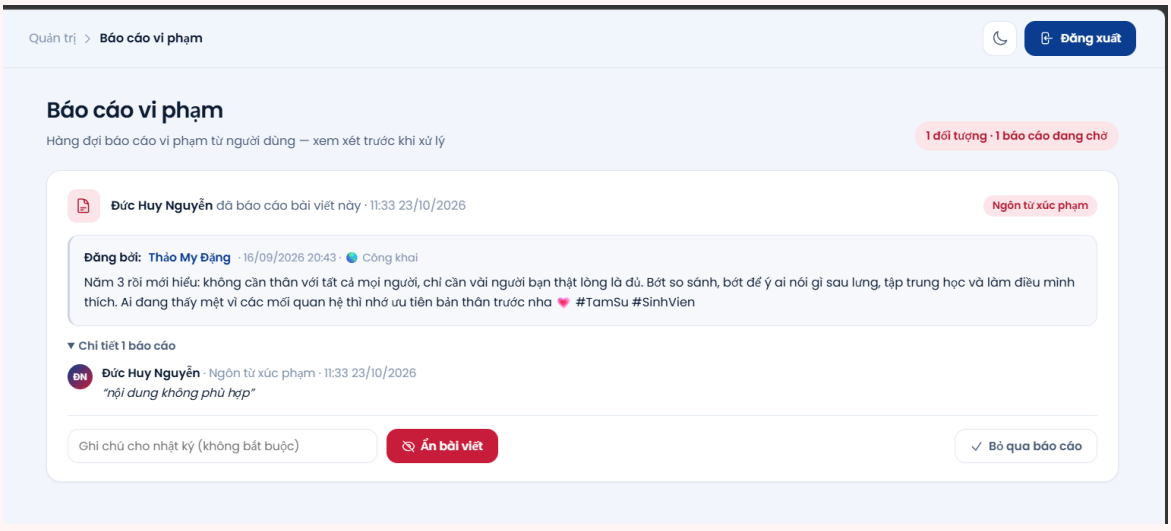


Figure 3.30. Handling violation reports



Figure 3.31. Admin audit logs

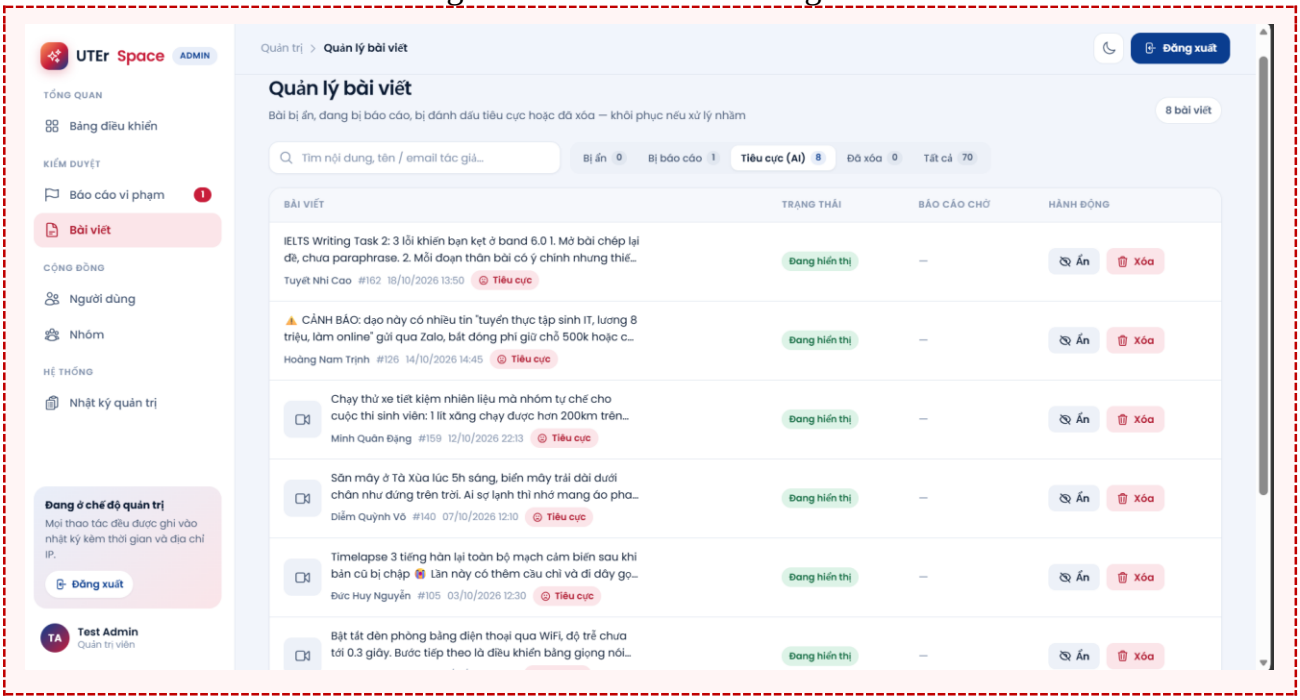


Figure 3.32. Post management with negative sentiment filter

3.16. Terms and Policies

The Terms of Service and Privacy Policy pages are publicly accessible to all visitors, and users are required to accept the terms during account registration.

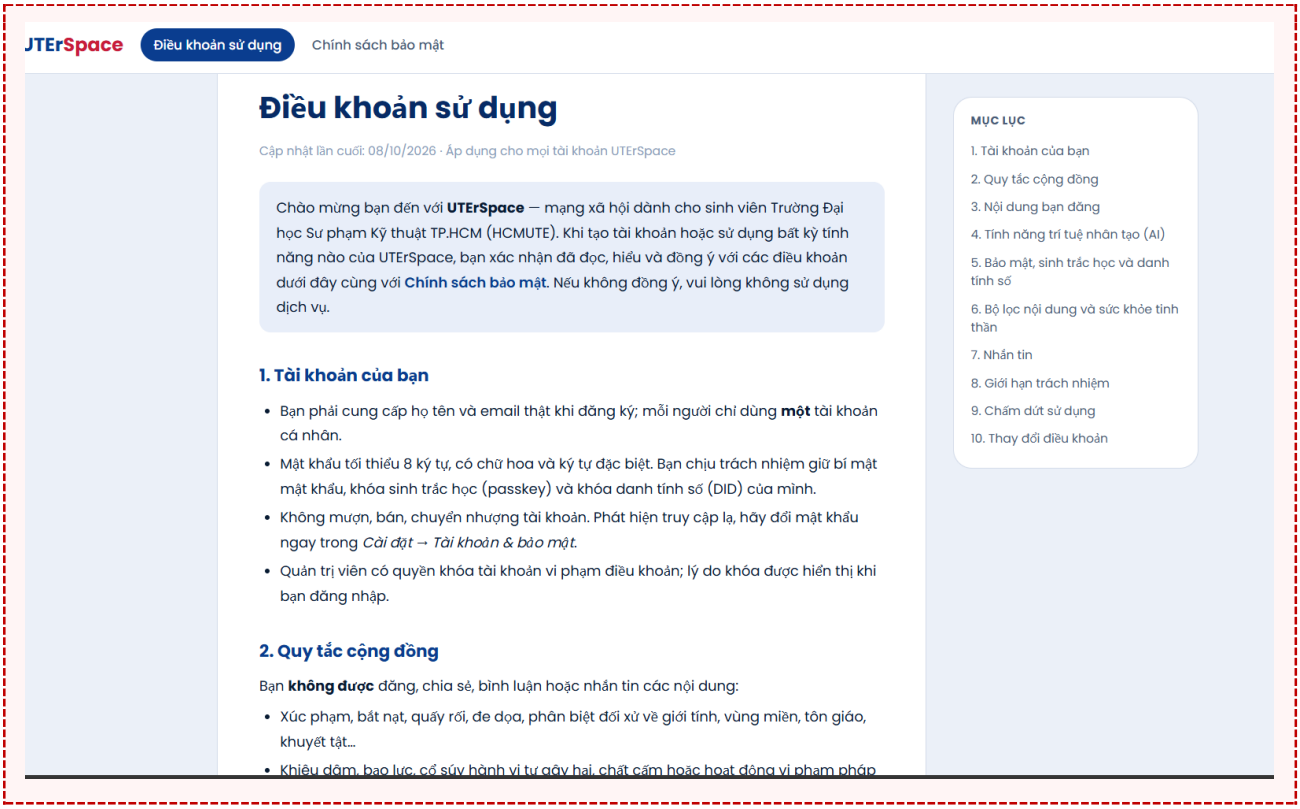


Figure 3.33. Terms of service page

CHAPTER 4. DATABASE

4.1. DBMS and Connection Method

UTErSpace uses MySQL 8.4.9 with the defaultdb database schema comprising 24 tables, configured with the utf8mb4 charset to fully support Vietnamese text and emojis. The MySQL database server is hosted on the team's machine on port 3306, accessible to all team members over Tailscale.

The application connects via JDBC through JPA. The database URL, MySQL8Dialect dialect, and HikariCP pool configurations are specified in the persistence.xml file, whereas database credentials are read by the CloudConfig class from Tomcat's external configuration file, remaining entirely outside the source repository. When Tomcat starts up, AppContextListener calls JPAUtil.init() to initialize the EntityManagerFactory; subsequently, each DAO acquires an EntityManager backed by a pre-warmed connection from HikariCP. Connection sessions are configured with the Asia/Ho_Chi_Minh time zone to guarantee timestamp consistency between database records and the application.

```
<properties>
  <!-- Database Connection: MySQL 8.4 trên Máy Chủ (Host Server qua Tailscale IP 100.119.255.40, Port 3306).
  | | | Tài khoản + mật khẩu đọc từ <thư mục Tomcat>/conf/uterspace-cloud.properties (JPAUtil.java). -->
  <property name="javax.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver" />
  <property name="javax.persistence.jdbc.url" value="jdbc:mysql://100.119.255.40:3306/defaultdb?useSSL=false&amp;allowPublicKe
  <!-- Pool kết nối HikariCP tối ưu cho MySQL 8.4 trên Host Server -->
  <property name="hibernate.connection.provider_class" value="org.hibernate.hikaricp.internal.HikariCPConnectionProvider" />
  <property name="hibernate.hikari.poolName" value="uterspace-db" />
  <property name="hibernate.hikari.minimumIdle" value="5" />
  <property name="hibernate.hikari.maximumPoolSize" value="20" />
  <property name="hibernate.hikari.connectionTimeout" value="20000" />
  <property name="hibernate.hikari.keepaliveTime" value="30000" />
  <property name="hibernate.hikari.maxLifetime" value="900000" />
  <!-- Hibernate Settings -->
  <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect" />
  <property name="hibernate.hbm2ddl.auto" value="none" />
  <property name="hibernate.show_sql" value="false" />
  <property name="hibernate.format_sql" value="false" />
</properties>
```

Figure 4.1. Connection configuration in persistence.xml

```
// Khởi tạo EntityManagerFactory (được gọi bởi AppContextListener khi web server start)
public static synchronized void init() {
    if (factory == null) {
        // HikariCP mặc định kiểm tra (1 lượt mạng ~100 ms tới DB cloud) mọi kết nối rảnh quá 0,5 giây trước khi
        // cho mượn → trong lúc đang duyệt (kết nối vừa dùng xong) bỏ qua bước này tới 10 giây.
        // Không để lâu hơn: máy ngủ / Windows tạm dừng tiến trình thì luồng ping nền (keepaliveTime) không chạy,
        // máy chủ cắt kết nối → kết nối rảnh lâu PHẢI được kiểm tra, nếu không request sẽ lỗi "Connection reset".
        // Đặt trước khi lớp HikariCP được nạp (giá trị đọc một lần khi nạp lớp).
        System.setProperty("com.zaxxer.hikari.aliveBypassWindowMs", "10000");
        factory = Persistence.createEntityManagerFactory("uterspacePU", overrides());
        LOG.info("[JPAUtil] Database: Đã kết nối tới " + factory.getProperties().get("javax.persistence.jdbc.url"));
    }
}
```

```
35      /** Tài khoản DB và địa chỉ / số kết nối ghi đè lên persistence.xml (nếu có khai báo trong CloudConfig) */
36      static Map<String, Object> overrides() {
37          Map<String, Object> props = new HashMap<>();
38          String user = CloudConfig.get(key: "db.user");
39          if (!user.isEmpty()) props.put("javax.persistence.jdbc.user", user);
40          String password = CloudConfig.get(key: "db.password");
41          if (!password.isEmpty()) props.put("javax.persistence.jdbc.password", password);
42          String url = CloudConfig.get(key: "db.url");
43          if (!url.isEmpty()) props.put("javax.persistence.jdbc.url", url);
44          String pool = CloudConfig.get(key: "db.poolSize");
45          if (!pool.isEmpty()) props.put("hibernate.hikari.maximumPoolSize", pool);
46          return props;
47      }
```

Figure 4.2. JPAUtil class initializing connections

4.2. Database Structure

The 24 database tables are categorized into four functional subsystems, as outlined in Table 4.1.

Subsystem	Tables	Description
Account & Security	users user_settings webauthn_credentials user_dids	User accounts, roles, lock status Privacy, notification, and filter settings Passkey public keys did:key digital identity
Posts & Interactions	posts comments likes saved_posts search_history	Posts, media, visibility scope, sentiment labels Comments Likes Saved posts Search history
Friends, Groups & Stories	friendships user_blocks groups_tbl group_members stories story_views story_reactions	Friendships Blocked users Groups Group members 24-hour stories Story views Story heart reactions
Messaging, Notifications & Administration	conversations conversation_members chat_messages message_reactions message_hidden notifications reports admin_logs	Direct and group conversations Members, delivered and read watermarks Messages, file attachments Message emoji reactions Locally hidden messages Notifications Violation reports Admin audit logs

Table 4.1. Tables in the defaultdb database

4.3. Subsystem Relationship Diagrams

4.3.1. Users and Security

Each user account has at most one row in user_settings and one row in user_dids, using user_id as the primary key in a one-to-optional-one (1:0..1) relationship; the settings row is only created when the user saves preferences for the first time. An account can register multiple Passkeys in webauthn_credentials and maintain multiple search_history entries. The user_blocks table references users twice—for the blocker and the blocked user—with a composite primary key spanning both columns. All foreign keys across this subsystem employ ON DELETE CASCADE, ensuring that deleting a user account cleanly cascades to remove all associated security and setting records.

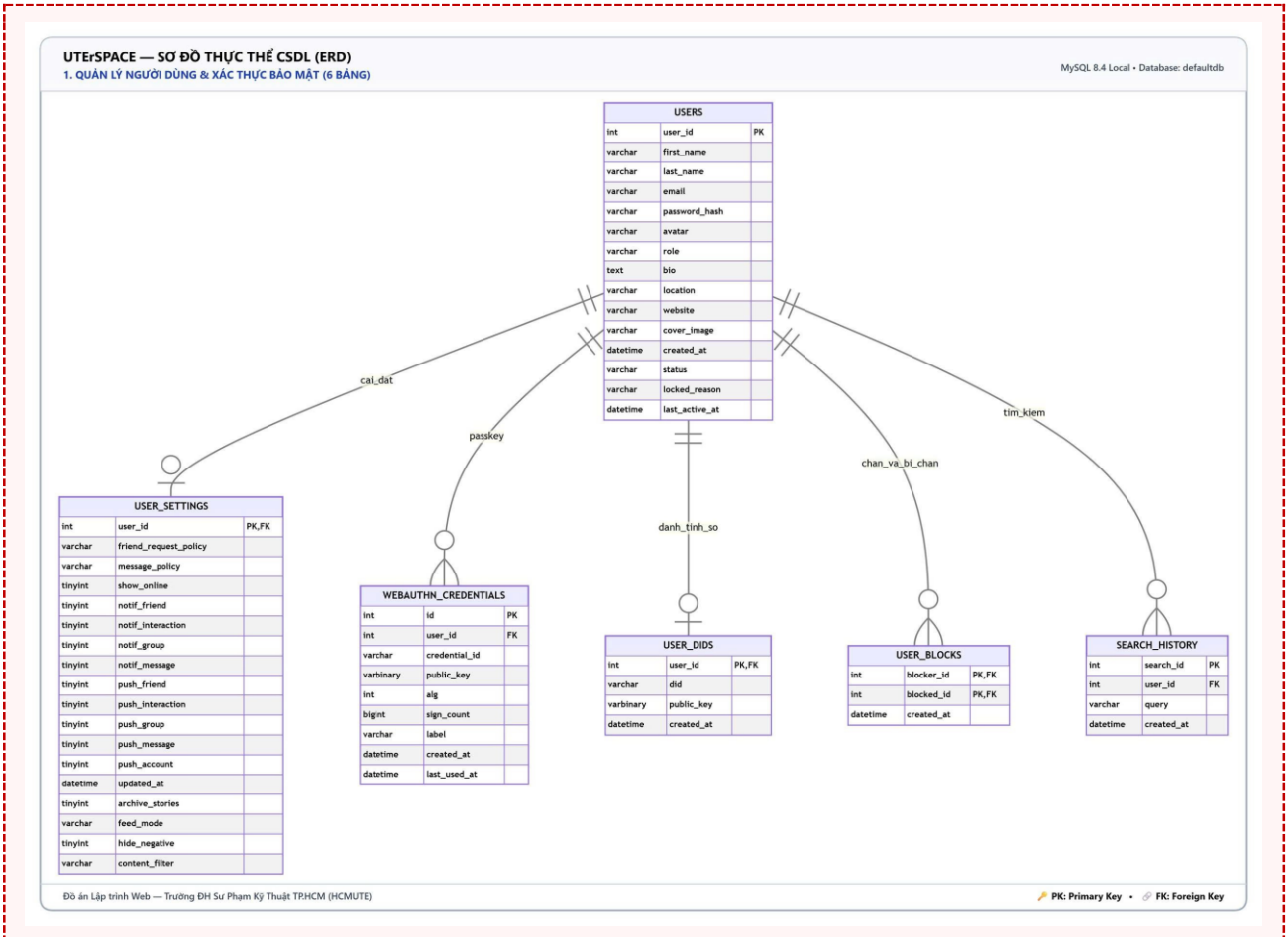


Figure 4.3. Users and Security subsystem diagram

4.3.2. Posts and Interactions

A user can publish many posts; each post can have multiple comments, likes, and bookmarks, with all three child tables referencing both posts and users concurrently. A UNIQUE constraint on the (post_id, user_id) tuple in likes and saved_posts guarantees that each user can like or save a post only once. Deleting a post cascades to remove its comments, likes, and saved entries. The posts.group_id column references groups_tbl with

ON DELETE SET NULL, whereas shared_post_id intentionally omits a foreign key constraint so that quote-shared posts remain intact even if the original post is deleted.

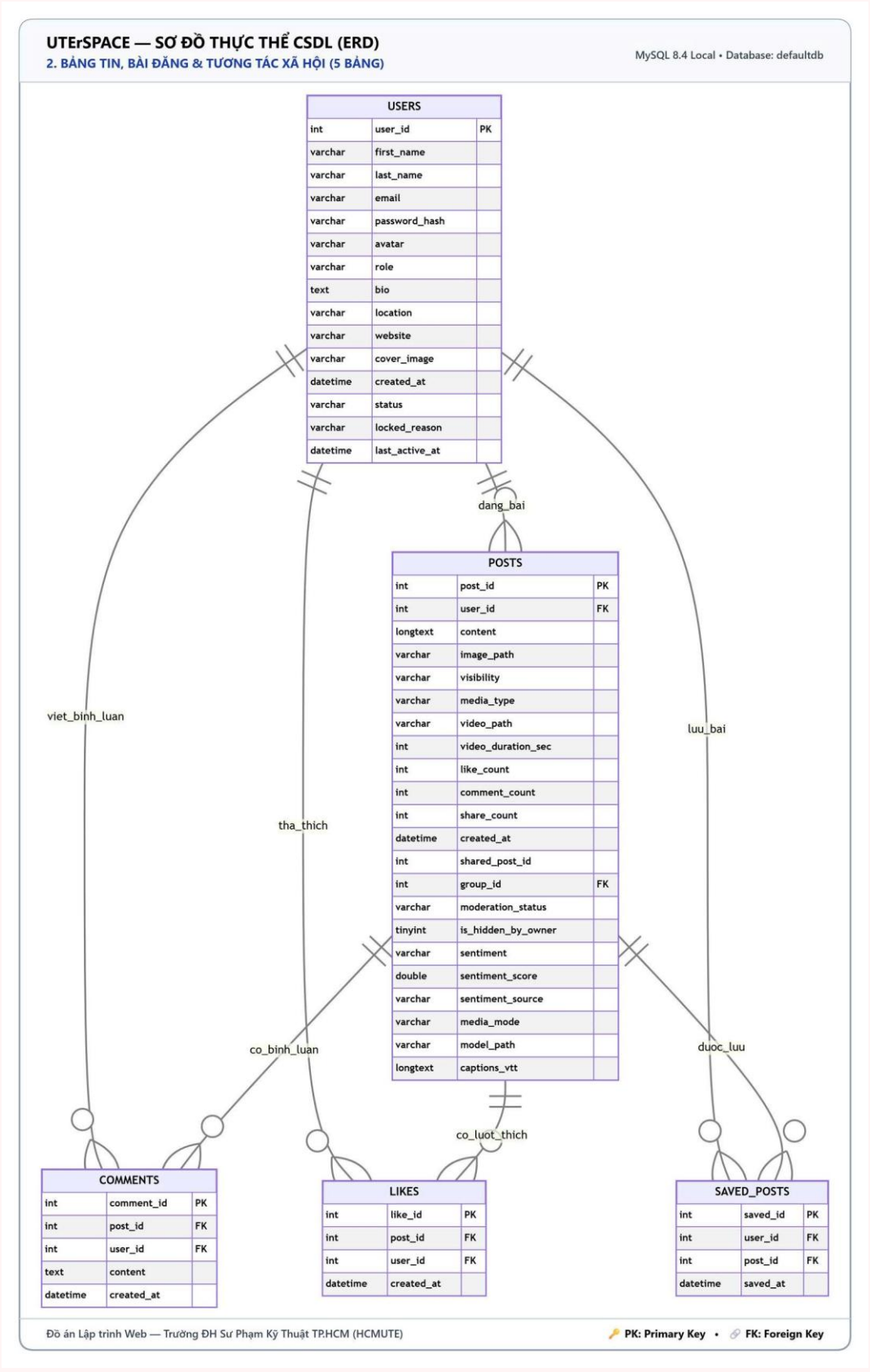


Figure 4.4. Posts and Interactions subsystem diagram

4.3.3. Friends and Groups

The friendships table references users twice—for the requester and the addressee. Two generated columns, user_lo and user_hi, automatically store the smaller and larger IDs of the user pair, guarded by a UNIQUE constraint on (user_lo, user_hi); thus, even if user A invites user B and user B invites user A concurrently, exactly one friendship row is created. The group_members table acts as a many-to-many junction table between users and groups_tbl with a composite primary key, tracking member roles and membership status.

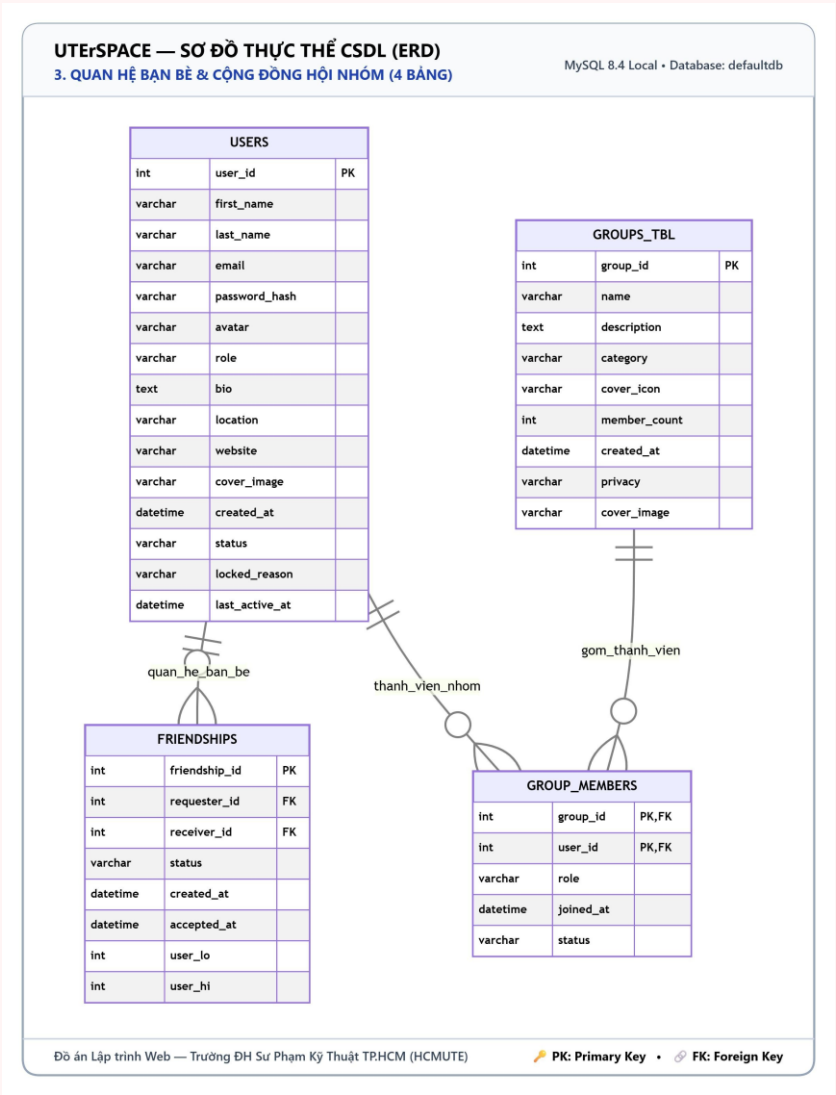


Figure 4.5. Friends and Groups subsystem diagram

4.3.4. Messaging and Notifications

A conversation contains multiple messages and multiple members. The conversation_members table uses a composite primary key on (conversation_id, user_id), tracking delivered watermarks, read watermarks, and per-user message history clear

timestamps. For 1-on-1 direct conversations, columns user_a_id and user_b_id guarded by a UNIQUE constraint ensure exactly one conversation exists per user pair; for group chats, these two columns are left NULL. The chat_messages table enforces a UNIQUE constraint on the sender and client-generated message UUID, preventing duplicate message delivery upon network retries. Emoji reactions and locally hidden message states are stored in message_reactions and message_hidden, both cascading on message deletion. The notifications table references users twice—for the recipient and the actor who triggered the event.

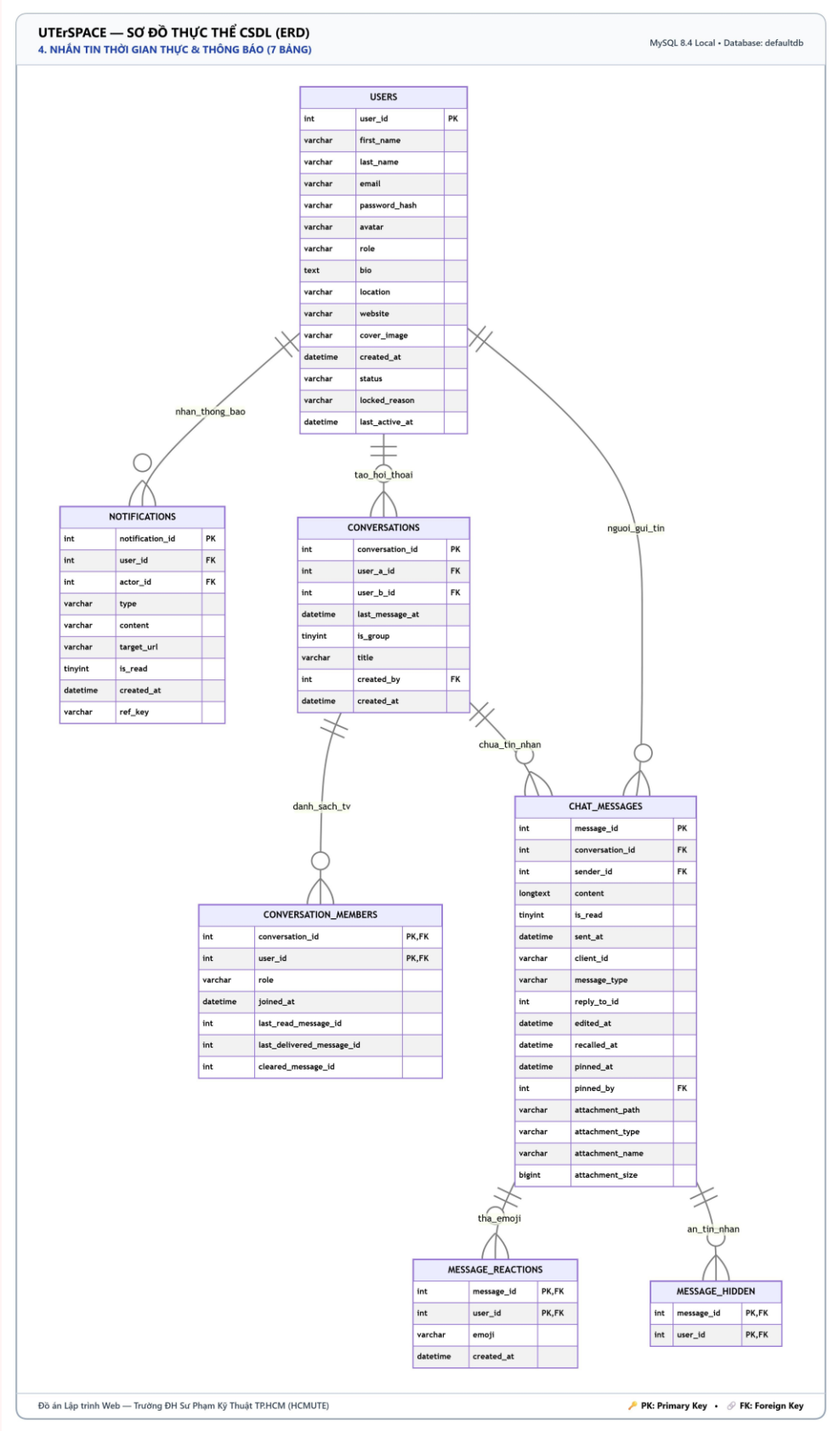


Figure 4.6. Messaging and Notifications subsystem diagram

4.3.5. Stories, Reports, and Administration

A user can share multiple stories; each story tracks multiple views and heart reactions, both tables using a composite primary key on (story_id, user_id) so each viewer is counted

only once, cascading upon story deletion. The reports table references the reporting user and the resolving administrator; the admin reference utilizes ON DELETE SET NULL to preserve audit history if an administrator account is removed, and a UNIQUE constraint on reporter and target entity prevents duplicate reports. The admin_logs table records every administrative action alongside its timestamp and IP address.

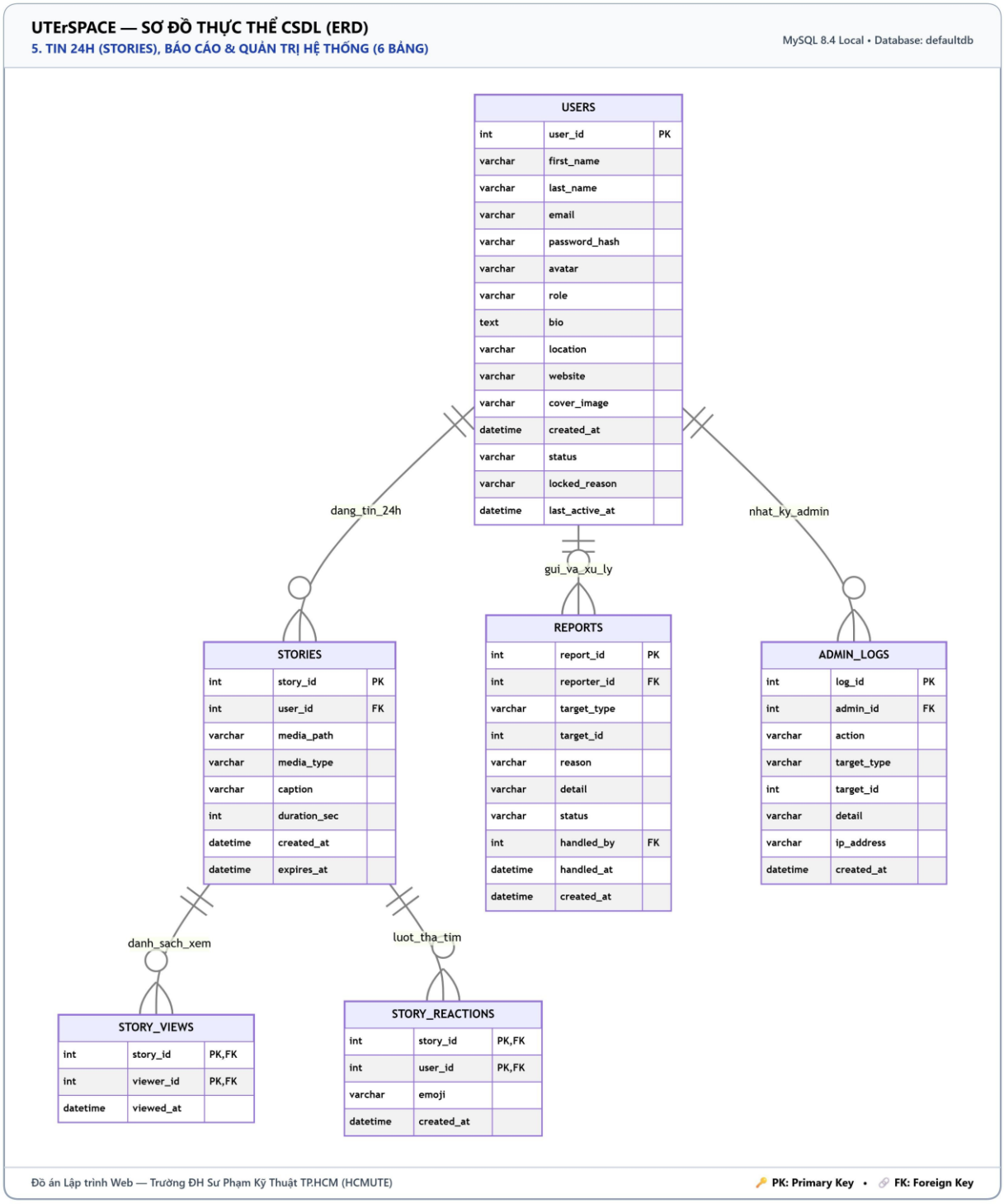


Figure 4.7. Stories, Reports, and Administration subsystem diagram

4.4. General Constraints

The schema consistently applies two deletion rules. Data completely dependent on a parent record—such as comments, likes, and messages—uses ON DELETE CASCADE to eliminate orphaned records; optional references—such as a post's group association, conversation creator, or message pin author—use ON DELETE SET NULL to preserve the underlying record. CHECK constraints enforce valid values for sentiment tags, media display modes, and feed display modes. The DID identifier and Passkey credential ID columns use the ascii_bin collation for strict case sensitivity, whereas all other text columns use utf8mb4.

4.5. Current Data

At the time of writing this report, the database contains 33 accounts (including 2 administrators), 69 posts with 21 videos, 239 comments, 536 likes, 11 groups, 42 friendships, 64 messages, and 1,001 notifications. Post images and videos are hosted on Cloudinary, with the database storing only their corresponding paths.

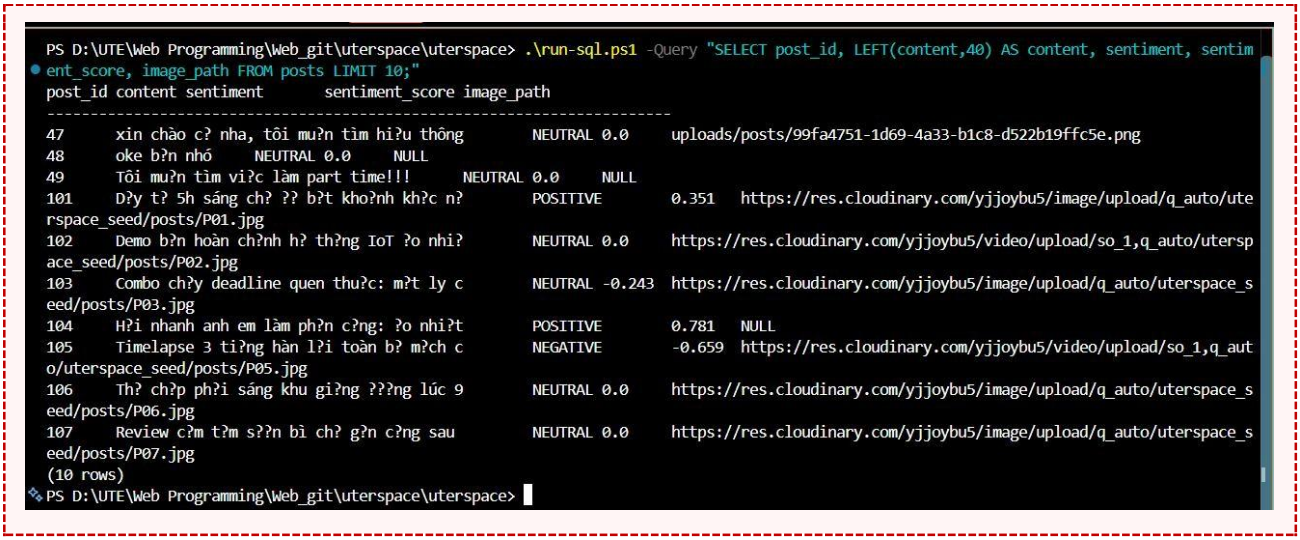


Figure 4.8. Post records with sentiment labels and Cloudinary paths



Figure 4.9. Row counts of major database tables

4.6. Backup and Recovery

The MySQL schema is incrementally updated via the aiven_mysql_migration.sql script, designed to be idempotently re-runnable without corrupting data. The entire

database is backed up into the uterspace_data_export.sql file using mysqldump and can be restored on another machine using the import-database.ps1 script.

CHAPTER 5. CONCLUSION

5.1. Results Achieved

The team has successfully built a fully functional social networking platform following the multi-tier MVC architecture on Tomcat 9, backed by a shared team database and CDN-hosted media. In addition to essential social networking features, the system successfully integrates WebSocket for real-time interactions, artificial intelligence for sentiment analysis and authoring assistance, accent-insensitive Vietnamese search, WebAuthn biometric authentication, and W3C DID decentralized digital identity.

5.2. Limitations

The primary database currently relies on a personal server; when this machine is powered off, team members must temporarily switch to an Aiven Cloud replica. The system does not yet feature an automated test suite and has not been deployed to an official production cloud hosting provider.

5.3. Future Development

In the future, the team aims to containerize the application using Docker and deploy it to a managed cloud hosting provider with automated database management and backups, introduce automated testing, integrate Web Push notifications, and implement real-time voice and video calling.

REFERENCES

[1] Oracle, Java Servlet 3.1 Specification (JSR 340) and Java API for WebSocket (JSR 356).

[2] Hibernate, Hibernate ORM 5.6 User Guide, https://hibernate.org/orm/documentation/5.6/

[3] Oracle, MySQL 8.4 Reference Manual, https://dev.mysql.com/doc/refman/8.4/en/

[4] W3C, Web Authentication: An API for accessing Public Key Credentials, Level 2, https://www.w3.org/TR/webauthn-2/

[5] W3C, Decentralized Identifiers (DIDs) v1.0, https://www.w3.org/TR/did-core/

[6] W3C, Verifiable Credential Data Integrity — EdDSA Cryptosuites, https://www.w3.org/TR/vc-di-eddsa/

[7] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text”, ICWSM, 2014.

[8] Cloudinary, Cloudinary Documentation, https://cloudinary.com/documentation

[9] Tailscale, Tailscale Documentation, https://tailscale.com/kb